NEUROSONIC
SOURCE CODE EXPLANATION & FACULTY VIVA HANDBOOK

How every important source file of the NeuroSonic prototype works, in simple English

Allwin Raj · Vellore Institute of Technology

Prepared 3 November 2026 from the actual code in `F:\Projects\WOAD-Research\implementation`

How this handbook was made. Every explanation was written after reading the actual source file. File names, function names, variable names, constants and line numbers are taken from the code as it is on 3 November 2026. Numbers in the results sections come from the project's own result files (`outputs/benchmarks/`, test logs).

Honesty rule. NeuroSonic is a research prototype, **not a safety device**. It does not measure distance, does not see in 360 degrees and does not guarantee safe navigation. Physical iPhone and field tests are still pending.

Table of Contents

How to Use This Handbook **5**

Part 1 – Project Architecture Overview **6**

1.1 What the system is 6

1.2 Block diagram 6

1.3 Folder map (only the important files) 6

Part 2 – Complete Application Execution Flow **8**

2.1 The ten steps from start-up to spoken warning 8

2.2 What one camera frame costs (measured on the laptop) 9

Part 3 – Individual Source Code Explanations **10**

A1. File: neurosonic_app.py 11

A2. Files: START_NEUROSONIC.ps1 and STOP_NEUROSONIC.ps1 13

A3. File: requirements.txt 14

B1. File: neurosonic/server.py 15

B2. File: neurosonic/service.py 17

B3. File: neurosonic/store.py 18

B4. Files: neurosonic/tls.py and neurosonic/netcheck.py 19

B5. File: neurosonic/tts.py 20

B6. File: neurosonic/voicecmd.py 21

B7. File: neurosonic/maps.py 22

B8. File: neurosonic/vision/assistant.py 23

B9. Files: neurosonic/vision/gemini.py, base.py, mock.py 24

C1. File: neurosonic/awareness.py 25

C2. File: neurosonic/detectors.py 27

C3. File: neurosonic/pathguide.py 29

C4. File: neurosonic/tracking.py 31

C5. File: neurosonic/events.py 32

D1. File: woad/model_loader.py 33

D2. Files: woad/arch/yolo.py and woad/arch/common.py 34

D3. Files: woad/preprocess.py and woad/ops.py 35

D4. File: woad/detector.py 36

D5. File: woad/quality.py 37

D6. File: woad/alerts.py 38

D7. Files: woad/sources.py, replay.py, pairing.py, visualize.py, metrics.py, woad_detect.py, woad_replay.py 39

E1. File: web/index.html 40

E2. File: web/style.css 41

E3. File: web/js/app.js 42

E4. File: web/js/speech.js 44

E5. File: web/js/nav.js 45

E6. Files: web/js/mapview.js, recorder.js, eventsim.js 46

F1. File: tools/benchmark_detectors.py 47

F2. File: tools/benchmark_precision.py 48

F3. File: tools/soak_test.py 49

F4. Files: tools/evaluate.py and tools/verify_equivalence.py 50

F5. Folder: tests/ (Python, Node.js and browser tests) 51

Part 4 – AI Engine and Model Explanation 52

4.1 RF-DETR Nano (primary detector) 52
4.2 SSDLite320-MobileNetV3 (fallback detector) 52
4.3 Choosing and switching detectors (GPU first, CPU fallback) 53
4.4 Which objects are reported 53
4.5 Thresholds and post-processing in one table 53
4.6 CUDA, FP16 and CPU fallback in simple words 54
4.7 The base-paper WOAD model (research mode) 54
4.8 Optional online scene description (not a detector) 54

Part 5 – Backend Explanation 55

5.1 What the backend is made of 55
5.2 How a request is handled 55
5.3 All API endpoints 55
5.4 Concurrency – who runs where 56
5.5 Data stored and privacy 56

Part 6 – Frontend Explanation 57

6.1 Technology 57
6.2 Camera loop on the phone – app.js 57
6.3 Speech priorities – speech.js 57
6.4 Accessibility features in the code 58

Part 7 – GPU and Python Environment 59

7.1 Hardware 59
7.2 Two Python environments (both Python 3.12.10) 59
7.3 How to check the GPU in front of the faculty 59
7.4 Two GPU details worth mentioning 59

Part 8 – Testing and Benchmarking 60

8.1 Automated tests (summary of Part 3, F5) 60
8.2 Benchmarks and what they proved 60
8.3 Detector comparison (Phase 1, RTX 4050) 60
8.4 What is NOT yet tested (say this honestly) 60

Part 9 – Important Code Snippets Explained Line by Line 62

9.1 Server start-up – neurosonic_app.py 62
9.2 Routing – server.py _dispatch 62
9.3 Camera frame endpoint – server.py r_awareness_frame 62
9.4 One inference thread – awareness.py 63
9.5 GPU detector with FP16 – detectors.py 63
9.6 Automatic fallback during a session – FallbackDetector.detect 63
9.7 Phone side – app.js awarenessStep 64

Part 10 – Faculty Viva Questions and Answers 65

10.1 Project and architecture 65
10.2 AI and models 65
10.3 Algorithms 66
10.4 Engineering and testing 66

Part 11 – Quick Revision Sheet **67**
11.1 Numbers to remember 67
11.2 The pipeline in one line 67
11.3 Which file does what 67
11.4 Honest limitations (always say them) 68
11.5 Demo checklist 68

How to Use This Handbook

When the professor points at a file: go to Part 3 and find that file. Each file has the same seven headings:

1. **Purpose** – what the file is for.
2. **How the code works** – the execution flow, step by step.
3. **Important functions and classes** – input, processing, output, and who calls them.
4. **Important code sections** – the blocks worth showing on screen.
5. **Connections** – which other files it talks to.
6. **Faculty demonstration explanation** – a short paragraph you can say aloud (yellow box).
7. **Possible faculty questions** – with short, correct answers.

When the professor asks a big-picture question ("how does the camera reach the AI?", "why the GPU?"), use Parts 1, 2, 4, 5, 6 and 7. Part 9 explains the most important code line by line. Part 10 is a viva question bank. Part 11 is a one-page revision sheet to read just before the demonstration.

Line numbers. References such as `awareness.py` L205 mean "line 205". In VS Code press `Ctrl+P`, type the file name, then `Ctrl+G` and the line number.

Words used in this handbook

Term	Simple meaning
Backend / server	The Python program on the laptop that does all the work
Frontend / client	The web page on the iPhone (HTML, CSS, JavaScript)
API endpoint	A web address such as `/api/awareness/frame` that the phone calls to ask the laptop to do something
JSON	A text format for structured data, e.g. `{"state": "CLEAR_VIEW"}`
Inference	Running a trained AI model on new input to get a prediction
Bounding box	A rectangle `[x1, y1, x2, y2]` around a detected object, in pixels
Confidence	The model's score between 0 and 1 for a detection
CUDA	NVIDIA's software that lets PyTorch run calculations on the graphics card (GPU)
FP16 / FP32	16-bit ("half") or 32-bit ("full") decimal numbers used inside the network
Thread	A separate line of execution inside one program, so several jobs can run at once
HTTPS / TLS	Encrypted web connection; iPhones only allow camera, microphone and GPS on HTTPS pages

Part 1 – Project Architecture Overview

1.1 What the system is

NeuroSonic has two halves that talk over the home Wi-Fi:

- **iPhone (thin client).** Safari opens a web page served by the laptop. The page uses the phone's rear camera, microphone, GPS and speaker. It does almost no computing; it captures, sends, displays and speaks.
- **Laptop (server).** One Python program, `neurosonic_app.py`, runs the web server and every engine: AI object detection (RF-DETR Nano on the NVIDIA RTX 4050 GPU, SSDLite on the CPU as fallback), walking-path rules, tracking, neural speech (Piper), speech recognition (Vosk), offline map routing (A*), history (SQLite), and an optional online scene description (Google Gemini). It also contains the re-implemented base-paper model, **WOAD**.

All local functions work **without internet**. Only the optional Gemini description needs it.

1.2 Block diagram

```
 iPhone Safari  (web/index.html + web/js/*.js)                    Laptop  (neurosonic_app.py)
 ──────────────────────────────────────────────────────────────────────────────────────────

 camera — 320 px JPEG every 200 ms ── HTTPS POST ──────────────►  server.py  r_awareness_frame
                                                                   └► awareness.py
RGBAwareness.process_frame
                                                                        ├ quality()          brightness
check
                                                                        ├ detectors.py       RF-DETR
(GPU, FP16) / SSDLite (CPU)
                                                                        ├ tracking.py        camera
motion + ByteTracker
                                                                        ├ pathguide.py       5 path
states
                                                                        └ events.py          entering /
crossing
                          ◄──────────── JSON (state, events, boxes) ─┘
 speaker ◄── WAV ──────────────────── GET /api/tts?text= ──────  tts.py  (Piper neural voice, cached)
 mic ── 16 kHz WAV ─────────────────── POST /api/voicecmd ──────►  voicecmd.py  (Vosk) → intent +
place
 GPS (stays on phone) ── from/to ──── POST /api/route ──────────►  maps.py  (A* on offline Vellore
graph)
 History screen ◄──────────────────── GET /api/history ─────────  store.py  (SQLite, text only)
 "Describe" (opt-in) ── 1 photo ──── POST /api/vision/describe ►  vision/assistant.py →
vision/gemini.py (internet)
 Research tools ──────────────────── /api/replay/*, /api/detect ► service.py → woad/*  (base-paper
WOAD model)
```

1.3 Folder map (only the important files)

Folder / file	What it contains
`neurosonic_app.py`	Entry point: reads command-line options, starts every engine and the server
`neurosonic/server.py`	Web server, all API routes, HTTPS handling

Folder / file	What it contains
`neurosonic/awareness.py`	Live camera pipeline (one frame in, path state and events out)
`neurosonic/detectors.py`	RF-DETR Nano (GPU, FP16) and SSDLite (CPU) detectors, automatic fallback
`neurosonic/pathguide.py`	Walking-path region and the five path states
`neurosonic/tracking.py`	Camera-motion estimate and ByteTrack-style object tracker
`neurosonic/events.py`	"Moving into your path" and "crossing in front" rules
`neurosonic/tts.py`, `voicecmd.py`, `maps.py`	Piper voice, Vosk voice commands, offline map and A* routing
`neurosonic/store.py`, `service.py`	SQLite history; WOAD replay/upload service and event log
`neurosonic/tls.py`, `netcheck.py`	Local certificate authority for HTTPS; Windows firewall check
`neurosonic/vision/`	Optional online scene description: `assistant.py`, `gemini.py`, `base.py`, `mock.py`
`woad/`	Base-paper WOAD model: loader, network (`arch/`), preprocessing, quality gate, alert engine
`web/`	Frontend: `index.html`, `style.css`, `js/app.js`, `speech.js`, `nav.js`, `mapview.js`, `recorder.js`, `eventsim.js`
`tools/`	Benchmarks, evaluation, soak test, figure and map builders (not used by the running app)
`tests/`	204 Python tests (including Node.js tests for the JavaScript) and browser tests
`START_NEUROSONIC.ps1`, `STOP_NEUROSONIC.ps1`	Start and stop the server as a background process
`models/`, `weights/`, `maps/vellore/`	Model files (binary), WOAD checkpoint copy, offline map data

Frameworks actually used: PyTorch (with torchvision and the `rfdetr` package), OpenCV, NumPy, ONNX Runtime (only inside Piper), Vosk, and the optional `google-genai`. **TensorFlow is not used**: it is not installed in either Python environment and no project file imports it. The web server uses only Python's standard library (`http.server`), and the frontend is plain JavaScript with no framework.

Part 2 – Complete Application Execution Flow

2.1 The ten steps from start-up to spoken warning

1. **The application starts.** `START_NEUROSONIC.ps1` runs `..\.venv-gpu\Scripts\python.exe neurosonic_app.py --host 0.0.0.0 --https` as a background process (through Windows WMI, so it keeps running when the terminal closes) and writes a log to `logs\neurosonic_<time>.log`.
2. **The Python environment is chosen.** `.venv-gpu` contains PyTorch 2.15.0 built for CUDA 13.2 plus `rfdetr` 1.11.2, so the GPU detector can load. With `-Cpu` the script uses `.venv` (CPU PyTorch), and the app automatically uses SSDLite instead.
3. **The backend server starts** (`neurosonic_app.py` `main()`, L28):
4. `ensure_certificates()` creates or reuses the local CA and server certificate (`tls.py`);
5. `Store` opens the SQLite database; `DetectionService` loads the WOAD checkpoint;
6. `PiperTTS.warm_up()`, `OfflineMap(...)`, `VoiceCommands.warm_up()` load the voice, map and speech model;
7. `RGBAwareness.warm_up()` loads RF-DETR on the GPU (or SSDLite) and runs one blank test frame;
8. `make_provider()` and `SceneAssistant` set up the optional Gemini description (no request is made);
9. `make_server()` opens port 8765 (HTTPS), `make_setup_server()` opens port 8766 (certificate setup page);
10. it prints `NeuroSonic running: …` and the iPhone address, then `serve_forever()` waits for requests.
11. **The frontend is loaded.** The iPhone opens `https://192.168.1.5:8765`. `server.py` `_static()` sends `web/index.html`, `style.css` and the JavaScript modules. `app.js` calls `init()` (`/api/status`) and starts `pollEvents()`, and fetches `/api/awareness/status`, `/api/tts/status`, `/api/map/info`, `/api/voicecmd/status`.
12. **Camera input is received.** The user taps **Start**. `startAwareness()` (app.js L541) asks for the rear camera with `getUserMedia({video: {facingMode: "environment", width 640, height 480}})`.
13. **Frames are processed and sent.** Every `recommended_frame_ms` (200 ms on the GPU, 400 ms on the CPU) `awarenessStep()` (L580) draws the video into a 320-pixel-wide canvas, encodes JPEG (quality 0.7) and sends it with `fetch("/api/awareness/frame?session=…&mode=chest&describe=0", {method: "POST", body: blob})`. Only one request is in flight at a time; a request that takes more than 5 s is cancelled.
14. **The AI model performs detection.** `server.py` `r_awareness_frame()` (L377) reads the JPEG and calls `RGBAwareness.process_frame()` (awareness.py L205), which decodes it with OpenCV, checks brightness, and runs the detector on the single inference thread. RF-DETR Nano returns boxes, class ids and confidences; only obstacle classes with confidence ≥ 0.40 are kept.
15. **Results are combined and returned.** The same function estimates camera motion and updates the tracker (`tracking.py`), decides the walking-path state (`pathguide.py`), checks for entering/crossing events (`events.py`), and returns JSON: `path.state`, `events` (sentences to speak), `detections`, `tracks`, `timing`, `detector`. The server also adds each spoken event to the history (`service.events.add` → `store.add_alert`).
16. **The user is informed.** `app.js` draws the path region and boxes on the video, updates the large state card (green "Path appears clear", orange "Caution", red "Stop"), and calls `voice.say(text, category)`. `speech.js` puts the sentence in a priority queue, fetches the Piper audio from `/api/tts?text=…` and plays it. A "Stop" message (category `urgent`) interrupts lower-priority speech such as a navigation instruction.
17. **Other features run alongside.**

- **Voice commands:** the 🎤 Speak button records 16 kHz WAV (`recorder.js`) → `/api/voicecmd` → Vosk text → intent (e.g. `navigate` + place).
- **Navigation:** place search `/api/map/search`, route `/api/route` (A*), then `nav.js` `followRoute()` turns GPS positions into spoken turn instructions.
- **History:** every spoken alert is stored as text in SQLite and shown on the History screen.
- **Online description (opt-in):** one photo → `/api/vision/describe` → Gemini → filtered spoken sentence.

2.2 What one camera frame costs (measured on the laptop)

Measurement (laptop browser with a fake camera; Wi-Fi not included)	Result
Frames sent per second with the GPU detector	4.91–4.95
Camera capture → analysed answer (final code)	median 67 ms, p90 73 ms
Event received → speech starts (neural voice)	about 257–260 ms
RF-DETR Nano FP16, paced at 5 frames/s (`benchmark_precision.py`)	median 53.2 ms (FP32: 66.5 ms)

Part 3 – Individual Source Code Explanations

Files are grouped as: **A** start-up and configuration, **B** backend, **C** AI engine and camera processing, **D** base-paper WOAD package, **E** frontend, **F** tools and tests. Line numbers (L…) refer to the code on 3 November 2026.

A1. File: neurosonic_app.py

1. Purpose

The main program of NeuroSonic. It reads the command-line options, creates every engine (database, WOAD model, voice, map, voice commands, camera detector, online description), starts the HTTPS web server and prints the addresses the iPhone should open.

2. How the code works

1. `ROOT` is the `implementation` folder; it is added to `sys.path` so `neurosonic` and `woad` can be imported.
2. `main()` (L28) defines the options with `argparse` (L29–53) and parses them.
3. With `--https`, `ensure_certificates()` creates/reuses the certificates and `server_ssl_context()` builds the TLS context (L58–65). If OpenSSL is missing it prints an error and returns exit code 2.
4. It creates, in order: `Store` (SQLite), `DetectionService` (WOAD), `PiperTTS` + `warm_up()`, `OfflineMap`, `VoiceCommands` + `warm_up()`, `RGBAwareness` + `warm_up()`, then `make_provider()` + `SceneAssistant` (L67–114). Each prints one status line, for example `Camera awareness: RF-DETR Nano … ready (cuda; frame interval 200 ms …)`.
5. `make_server(...)` (L116) receives all engine objects. If the port is busy, it prints `ERROR: cannot listen … (is the app already running?)` and returns 2.
6. In LAN + HTTPS mode it starts the setup page server on port+1 in a thread (L126–129).
7. It prints `NeuroSonic running: …`, the phone URLs and the CA fingerprint, runs the Windows firewall check (`windows_report`), and finally calls `httpd.serve_forever()` (L146). On Ctrl+C the `finally` block stops the replay, closes the servers and the database.

3. Important functions and classes

Name	Input	Processing	Output	Called by
`main(argv=None)` L28	command-line options	builds all engines, starts servers	exit code 0 / 2	`if __name__ == "__main__"` (L159), `START_NEUROSONIC.ps1`

Key options: `--host` (default `127.0.0.1`; `0.0.0.0` for the phone), `--port` (8765), `--https`, `--detector` (`auto`/`rfdetr`/`ssdlite`, default from `NEUROSONIC_DETECTOR` or `auto`), `--vision-provider` (`gemini`/`mock`/`off`), `--vision-model` (default `gemini-3.8-flash`), `--no-events`, `--db`, `--map`, `--asr-model`, `--voice-model`, `--rgb-weights`.

4. Important code sections

- L93–94: `RGBAwareness(a.rgb_weights, detector=a.detector or os.environ.get("NEUROSONIC_DETECTOR") or "auto", …)` – "auto" means *try RF-DETR on the GPU first, otherwise SSDLite*.
- L104–105: `provider = make_provider(a.vision_provider, a.vision_model)` and `SceneAssistant(provider, enabled=…)` – this only configures Gemini; no internet request is made at start-up.
- L116–121: the `try/except OSError` around `make_server` – the friendly "already running" message.

5. Connections

Imports `neurosonic.server`, `service`, `store`, `tls`, `netcheck`, `tts`, `maps`, `voicecmd`, `awareness`, `vision` and `woad.model_loader`. It is started by `START_NEUROSONIC.ps1`.

6. Faculty demonstration explanation

> "This is the entry point. When I run it, it loads every engine one by one and prints one status line for each, so I can see immediately that the GPU detector, the voice, the map and the speech recogniser are ready. Then it starts an HTTPS server on port 8765 for the iPhone and a small setup page on 8766 for installing our certificate."

7. Possible faculty questions

- **Why `--host 0.0.0.0`?** It listens on all network interfaces, so the iPhone on the same Wi-Fi can connect; the default `127.0.0.1` accepts only the laptop itself.
- **What happens if the GPU is not available?** `RGBAwareness` falls back to SSDLite on the CPU and the start-up line prints the reason (`fallback: …`).
- **Does it call Gemini at start-up?** No. It only builds the provider object; a request is sent only when the user allows it and taps Describe.
- **Why exit code 2?** To signal a start-up failure (certificate or port problem) to the start script.

A2. Files: START_NEUROSONIC.ps1 and STOP_NEUROSONIC.ps1

1. Purpose

PowerShell scripts that start the server as an independent background process and stop it safely. They make the demonstration reliable: the server keeps running even if the terminal (or Claude Code) is closed.

2. How the code works (START)

1. Parameters: `-Port 8765`, `-Cpu`, `-VisionProvider`, `-VisionModel`, `-TimeoutSeconds 180`.
2. Chooses the interpreter: `..\.venv-gpu\Scripts\python.exe` (or `..\.venv\…` with `-Cpu`).
3. **Safety checks:** refuses to start if port 8765 or 8766 already accepts a TCP connection, or if any `neurosonic_app.py` process from the project environments is running (it names the process).
4. Creates the process through WMI (`Win32_Process.Create`) running `cmd.exe /d /c "python -u neurosonic_app.py --host 0.0.0.0 --https --port N > logs\neurosonic_<time>.log 2>&1"`, so the server is not a child of the calling shell.
5. Writes the PID to `data\neurosonic.pid`, waits until the log contains `NeuroSonic running:`, then calls `/api/awareness/status` and prints detector, device, precision, frame interval, the LAN IP and the iPhone URL.

STOP finds only processes whose command line contains `neurosonic_app.py` and whose executable is under `F:\Projects\WOAD-Research` (plus their child process and the `cmd` wrapper), stops them, and removes a stale PID file. `-WhatIf` lists without stopping; `-Port N` limits it to one server. Other Python programs are never touched.

3–5. Functions, sections, connections

The scripts call `neurosonic_app.py` and read `logs\` and `/api/awareness/status`. The important design point is the TCP-connect port check (Windows' "Listen" query returned nothing on this laptop, so the first version allowed a duplicate start; this was found and fixed).

6. Faculty demonstration explanation

> "To run the full application I use one script. It checks that nothing else is using the port, starts the server in the background, waits until the server reports it is listening, and then prints which detector is active and the address for the iPhone. The stop script stops only NeuroSonic, never other Python programs."

7. Possible faculty questions

- **Why WMI instead of `Start-Process`?** A process created by WMI is not part of the terminal's process tree, so closing the terminal does not kill it.
- **Where are errors shown?** In `logs\neurosonic_<time>.log` (the server's standard output and error).
- **Can two servers run at once?** Not on the same port: the script refuses, and the server itself binds the port exclusively (`SO_EXCLUSIVEADDRUSE` in `server.py`).

A3. File: requirements.txt

1. Purpose

Lists the exact package versions the tests passed with, so the environment can be rebuilt.

2–4. Content

`torch==2.15.0`, `torchvision==0.29.2`, `numpy==2.5.4`, `opencv-python==5.0.0.93`, `piper-tts==1.8.0`, `onnxruntime==1.31.1`, `vosk==0.3.45`, `google-genai==2.34.0`. Comments explain licences and that the web server, SQLite store, map routing and TLS use only the standard library (plus `numpy`, and the `openssl` command for HTTPS). The GPU environment `.venv-gpu` additionally has `torch 2.15.0+cu132`, `torchvision 0.29.1+cu132` and `rfdetr 1.11.2` (and `ultralytics`, used only for benchmarking).

6. Faculty demonstration explanation

> "All versions are pinned. The production environment `.venv` is CPU-only; the GPU work happens in a separate environment `.venv-gpu` so the working CPU setup was never put at risk."

7. Possible faculty questions

- **Why is TensorFlow not listed?** The project does not use it; all models run on PyTorch (Piper uses ONNX Runtime).
- **Why two environments?** Installing the 2.4 GB CUDA build into the working environment could break it; a separate one can be deleted without harm.

B1. File: neurosonic/server.py

1. Purpose

The web server of NeuroSonic. It serves the frontend files and all API endpoints (camera frames, speech, voice commands, map, history, online description, WOAD research tools), over HTTPS. It uses only Python's standard library.

2. How the code works

1. `make_server()` (L462) creates a `NeuroSonicHTTPServer` with a request-handler class built by `make_handler()` (L62). The engine objects (`service`, `store`, `tts`, `offline_map`, `voicecmd`, `awareness`, `vision`) are captured in that class, so every route can use them.
2. `NeuroSonicHTTPServer` (L421) extends `ThreadingHTTPServer`: **each connection is handled in its own thread**.
3. `finish_request()` (L444) runs in that worker thread: it performs the **TLS handshake** with a 10-second timeout (`TLS_HANDSHAKE_TIMEOUT`), then runs the handler.
4. `do_GET` / `do_POST` (L404–411) call `_dispatch()` (L133), which looks up the path in the `GET` or `POST` dictionary (L392–402). Unknown GET paths go to `_static()` (the web files).
5. The route method reads the body, calls the engine, and replies with `_json()` or `_send()`.
6. `_dispatch()` turns errors into HTTP status codes: `ApiError` → its own code (400/404/413/415/503), `TTSError` → 503, `ValueError` / `InvalidInput` → 400, client disconnect → quiet close, anything else → 500.

3. Important functions and classes

Name	Input	Processing	Output	Called by
`make_handler()` L62	engine objects	defines the `Handler` class and route tables	handler class	`make_server`
`Handler._raw_body(limit, kinds)` L91	binary upload	checks length (413), Content-Type (415)	bytes	frame, voice, vision routes
`Handler._body()` L103	JSON request	parses, checks it is an object	`dict`	JSON routes
`Handler._drain_unread_body()` L117	—	reads a rejected body so Windows does not reset the socket	—	`_dispatch` on error
`Handler._static(path)` L161	URL path	blocks path traversal, sets explicit MIME types (`STATIC_TYPES`)	file bytes	`_dispatch`
`r_awareness_frame()` L377	JPEG body, `session`, `mode`, `describe`	calls `awareness.process_frame`, logs events	JSON result	phone `awarenessStep()`
`r_tts()` L249 / `_send_media()` L255	`text`	Piper WAV; supports HTTP Range (206/416) for iOS	`audio/wav`	`speech.js`
`r_voicecmd()` L331	16 kHz WAV	`voicecmd.handle`	heard text, intent, places	`app.js listenOnce`
`r_route()` L318, `r_map_search()` L311	from/to or query	`offline_map.route` / `search`	route JSON / results	`app.js`
`r_vision_describe()` L356	photo, consent, question	`vision.describe`; deletes the image	spoken text JSON	`app.js describeScene`

Name	Input	Processing	Output	Called by
`NeuroSonicHTTPServer.finish_request()` L444	accepted socket	TLS handshake per connection, in the worker thread	—	`socketserver`
`server_bind()` L435	—	`SO_EXCLUSIVEADDRUSE` on Windows	—	server start
`make_setup_server()` L469	CA file	plain-HTTP page that only serves the CA certificate	server	`neurosonic_app.py`

4. Important code sections

- **Route tables (L392–402).** One place lists every endpoint: 16 GET routes and 12 POST routes.
- **Per-connection TLS (L444–459).** `request.settimeout(self.handshake_timeout)` then `self.ssl_context.wrap_socket(request, server_side=True)`. A failed or slow handshake drops only that connection.
- **Exclusive port (L433–438).** `allow_reuse_address = sys.platform != "win32"` and `SO_EXCLUSIVEADDRUSE`.
- **Image never stored (L366).** `del img` after the online description.

5. Connections

Called by the browser (`app.js`, `speech.js`). Calls `service.py`, `store.py`, `tts.py`, `maps.py`, `voicecmd.py`, `awareness.py`, `vision/assistant.py`. Created by `neurosonic_app.py`.

6. Faculty demonstration explanation

> "This is the backend. All endpoints are in one table at line 392. When the phone sends a camera frame to `/api/awareness/frame`, the server reads the JPEG, gives it to the awareness pipeline and returns JSON. Each connection runs in its own thread and does its own HTTPS handshake – that fixed a real bug where one slow iPhone connection froze the whole server."

7. Possible faculty questions

- **Why not Flask or FastAPI?** The standard library was enough, adds no dependencies, and gave low-level control needed for two real fixes (per-connection TLS, exclusive port binding).
- **Is it REST?** It is a resource-style HTTP/JSON API (GET reads, POST actions/uploads, status codes for errors); not strictly RESTful everywhere (e.g. `POST /api/places/delete`).
- **How are big uploads prevented?** `_raw_body` limits frames to 3 MB and WAV to 2 MB (413 otherwise); JSON bodies to 20 MB (`MAX_BODY`).
- **Why the `_drain_unread_body` function?** On Windows, closing a socket with unread data sends a reset, so the client saw "connection aborted" instead of the error message. Reading the body first fixed it (regression test).
- **Is there a login?** No. Anyone on the same Wi-Fi can reach it; this is a stated limitation.

B2. File: neurosonic/service.py

1. Purpose

Wraps the base-paper **WOAD** detector for the Research tools: a background **replay** of recorded IR + depth frames, and detection on a **single uploaded pair**. It also holds the **event log** that feeds the History.

2. How the code works

1. `DetectionService.__init__` (L165) loads `WOADDetector(weights, device, conf_thres=0.25)`, an `InputQualityChecker`, a `model_lock`, and an `EventLog(store)`.
2. `start_replay(name, fps)` (L184) checks the set name (only server-known sets in `REPLAY_SETS`) and fps (0.5–10), stops any old replay and starts a `ReplayRunner` thread.
3. `ReplayRunner.run()` (L124), for each frame: quality check → (if usable) WOAD detection under `model_lock` → `AlertEngine.process` → events added to the log → annotated JPEG published → wait for the next frame time.
4. `detect_upload()` (L213) decodes the two images, checks quality, runs WOAD once, and builds alerts with a single-frame `AlertConfig(confirm_hits=1, …)`.
5. `EventLog.add()` (L77) gives each event an increasing id (for `/api/events?since=`) and **writes it to SQLite** through `store.add_alert` – this is how camera alerts also reach the History.

3. Important functions and classes

Name	Input	Output	Called by
`decode_image(data, role)` L41	bytes	8-bit BGR image (rejects 16-bit / grey depth)	`detect_upload`
`EventLog.add / since / last_id` L77–96	source, kind, text, data	event dict; list of newer events	server routes, replay
`ReplayRunner(threading.Thread)` L99	set name, fps	state + JPEG (via `snapshot()`)	`start_replay`
`DetectionService` L164	store, weights, device	replay control, upload results	`server.py`

6. Faculty demonstration explanation

> "This file connects the base paper to the app. WOAD needs an infrared camera and a depth camera, which we do not have, so the replay plays the authors' recorded sensor frames through exactly the same path a live sensor would use: quality check, WOAD model, alert engine, spoken alerts."

7. Possible faculty questions

- **Why a lock around the model?** The replay thread and uploads share one model instance; the lock prevents two threads from running it at the same time.
- **Can a user replay any folder?** No. Only the names in `REPLAY_SETS` are accepted, so no client-supplied paths.
- **Where do events go?** Into an in-memory ring (300 items) for polling, and into SQLite for the History.

B3. File: neurosonic/store.py

1. Purpose

Local SQLite database for the **alert history** and **saved places**. Nothing leaves the laptop, and no image is ever stored – only text.

2. How the code works

1. `Store.__init__` (L33) opens the database file (`data/neurosonic.sqlite3`) with `check_same_thread=False` and a `threading.Lock`, and creates the tables from `SCHEMA` (L6) if they do not exist.
2. `add_alert()` (L47) checks `source` and `kind` against whitelists (`ALLOWED_SOURCES`, `ALLOWED_KINDS`, L28–29), cuts text to 300 characters and inserts a row using `?` parameters.
3. `history(limit)` returns the newest rows (1–1000); `stats()` counts by class, kind and source; `clear_history()` deletes all alerts.
4. `add_place/places/delete_place` manage saved places with latitude/longitude range checks.

4. Important code section

Table `alerts(id, created, source, kind, text, class_name, position, confidence, frame_id)` – note there is **no image column**. Table `places` has SQL `CHECK` constraints on `lat` and `lon`.

6. Faculty demonstration explanation

> "History is stored locally in SQLite, text only. Every insert uses parameters, so it is safe against SQL injection, and the source and kind are checked against a whitelist."

7. Possible faculty questions

- **Why SQLite?** Built into Python, a single file, no server, enough for one user.
- **Is it thread-safe?** All database calls run under one lock, because the web server uses many threads.
- **Are camera images stored?** No; the schema has no image field, and the frame bytes are discarded after analysis.

B4. Files: neurosonic/tls.py and neurosonic/netcheck.py

1. Purpose

`tls.py` creates the HTTPS certificates that an iPhone accepts. `netcheck.py` checks the Windows network profile and firewall rules and prints the exact fix commands.

2. How the code works (tls.py)

1. `ensure_certificates(tls_dir)` (L86): if `neurosonic-ca.crt/.key` do not exist, run `openssl req -x509 …` to make the local root CA "NeuroSonic Local CA" (825 days).
2. Collect the laptop's addresses with `lan_ips()` (L36) and the host name.
3. If the existing server certificate does not cover these addresses, belongs to another CA, or expires within 14 days, create a new one signed by the CA with `subjectAltName = IP:…, DNS:localhost, …`, `extendedKeyUsage = serverAuth`, `CA:FALSE`, 397 days – the rules iOS requires.
4. `server_ssl_context()` (L133) builds an `ssl.SSLContext` with minimum TLS 1.2.

6. Faculty demonstration explanation

> "Phones only allow the camera and GPS on HTTPS. A plain self-signed certificate was rejected by iOS, so this file creates a small local certificate authority. I install it once on the iPhone from the setup page; after that the connection is trusted and browser security is never disabled."

7. Possible faculty questions

- **Why did the first certificate fail on iOS?** It had no Subject Alternative Name and no serverAuth usage, and was marked as a CA; iOS 13+ rejects such server certificates.
- **What if the laptop's IP changes?** `ensure_certificates` notices the new address and issues a new server certificate automatically.
- **Is the private key safe?** It stays in `data/tls/` on the laptop and must never be uploaded (e.g. to GitHub).

B5. File: neurosonic/tts.py

1. Purpose

Offline **neural text-to-speech** with Piper. It turns a sentence into a WAV file on the laptop, which the phone plays. Voice: Indian-English `en_IN-spicor.onnx` (22.05 kHz mono).

2. How the code works

1. `PiperTTS.__init__` (L32) stores the model path, speaking `STYLE` (`length_scale 1.08`, `noise_scale 0.6`, `noise_w_scale 0.7`), and a cache folder.
2. `warm_up()` (L75) loads the model with `PiperVoice.load()` and synthesises "Ready." to prove it works.
3. `synthesize(text)` (L95): `clean()` collapses spaces and rejects empty text or more than 400 characters (`MAX_CHARS`); a SHA-256 key of voice + style + text is computed; the audio is returned from the in-memory cache (64 items), the disk cache (400 files), or generated with `voice.synthesize_wav()` under a lock and saved.

5. Connections

Called by `server.py` `r_tts()` (`GET /api/tts?text=`), which `speech.js` fetches. Started by `neurosonic_app.py`.

6. Faculty demonstration explanation

> "The voice is generated on the laptop by a neural model, so it works without internet. Common warnings like 'Stop' are cached, and the phone even fetches them in advance, so an urgent warning is never delayed by synthesis."

7. Possible faculty questions

- **What if Piper fails?** `TTSError` becomes HTTP 503 and the phone falls back to its built-in `speechSynthesis`.
- **Why cache by hash?** The same sentence always gives the same audio; hashing voice+style+text gives a safe file name.
- **Licences?** Piper is GPL-3.0-or-later; the voice is AGPL-3.0 (stated in `info()`).

B6. File: neurosonic/voicecmd.py

1. Purpose

Offline **voice commands** with Vosk (Indian-English small model). It converts a short recording into text and an *intent* such as `navigate` + "CMC Hospital".

2. How the code works

1. `handle(wav_bytes, near, saved)` (L224) checks the WAV (`read_wav16k`: mono, 16-bit, 16 kHz, ≤ 8 s) and rejects recordings shorter than 0.3 s.
2. `transcribe(pcm)` (L184) runs **two recognisers** on the same audio: one restricted to `COMMAND_WORDS` (robust for commands) and one free dictation (for place names).
3. `parse_intent(grammar_text, free_text)` (L74) uses regular expressions and word sets to choose one of 13 intents (navigate, search, start/stop navigation, repeat, status, awareness on/off, help, test voice, describe_scene, ask_scene, unknown).
4. For navigate/search, `resolve_place()` (L199) tries saved places, then the map's text search, then a **sound-alike** match using `phonetic()` (L45) and `place_score()` (L62), because Vosk cannot spell most Vellore names ("Katpadi" heard as "cut body").

6. Faculty demonstration explanation

> "Speech recognition runs offline on the laptop. One recogniser only knows the command words, the other listens freely for the place name, and a sound-alike matcher finds the closest place in the offline map. Because this is not fully reliable, the app says which place it chose and waits for 'start navigation'."

7. Possible faculty questions

- **Why two recognisers?** A grammar-restricted recogniser is reliable for fixed commands but cannot hear unknown names; free dictation hears names but often garbles command words.
- **How accurate are place names?** Correct first time in 10 of 24 test commands – so confirmation is required.
- **Is audio stored?** No; it is processed in memory and only the intent is logged.

B7. File: neurosonic/maps.py

1. Purpose

Offline map services on the local OpenStreetMap extract of Vellore (`maps/vellore/`): coverage check, place search and walking routes with the **A*** shortest-path algorithm, plus spoken turn instructions.

2. How the code works

1. `OfflineMap.__init__` (L59) loads `manifest.json`, `graph.json` (nodes, ways, edges) and `places.json`, builds an adjacency list, and finds **connected components** (islands such as paths inside Vellore Fort).
2. `search(query, near)` (L111) scores names: exact 100, prefix 80, contains 60, all words 40; ties by distance.
3. `route(start, end)` (L140): checks coverage, snaps both points to the nearest graph node (`nearest_node`, max 250 m, 600 m for islands), runs `_astar()`, rebuilds the path, and `_steps()` turns it into instructions.
4. `_astar(s, t)` (L184): a priority queue (`heapq`) ordered by *distance so far + straight-line (haversine) distance to the goal*. The straight-line estimate never over-estimates, so A* finds the shortest path.
5. `_steps()` (L222) keeps a turn only if the direction changes ≥ 30° (measured 20 m before/after the junction) or a named street begins, and merges steps shorter than 12 m.

6. Faculty demonstration explanation

> "Routing works without internet. The Vellore street network is a graph of 31,439 path points; A* searches it using straight-line distance as the guide. The output is a polyline for the map plus a short list of spoken turns, for example Katpadi to CMC Hospital gives six instructions."

7. Possible faculty questions

- **Why is the haversine heuristic valid?** No walking path can be shorter than the straight line, so it never over-estimates (it is *admissible*), which guarantees the shortest route.
- **Limits?** OSM data for Vellore is sparse: few street names, no pavement/crossing information, no buildings.
- **What if the destination is outside the map?** `MapError` with a clear sentence (HTTP 400).

B8. File: neurosonic/vision/assistant.py

1. Purpose

The provider-independent logic of the optional **online scene description**: consent, rate limits, retries, a circuit breaker, and a **safety filter** that removes distances and "it is safe" claims before anything is spoken.

2. How the code works (`SceneAssistant.describe`, L286)

1. Checks, in order: enabled → **consent** → provider configured → breaker not open → at most one request per 5 s per session (`min_interval_s`) → at most 60 per hour (`hourly_cap`).
2. `prepare_image()` (L174) validates the photo and shrinks it to ≤ 1024 px JPEG, **in memory only**.
3. `build_prompt()` adds the carrying mode and the question; `SYSTEM` (L69) tells the model the rules (positions only as left/centre/right, never distances, never "safe").
4. Calls `provider.analyze(req, prompt, SYSTEM, self.per_try_timeout_s)` – always the **full** 30 s attempt. Only a quick `ProviderUnavailable` (503/network) is retried once, and only if another full attempt fits in the 45 s budget. Timeouts are never retried.
5. On success `compose_speech()` (L121) builds "AI description: …", passing every part through `filter_text()` (L99), which drops sentences matching `_BANNED` patterns, and adds "Please check the path yourself." if anything was removed. On failure it returns a ready-to-speak text from `TEXT`, e.g. "The online description took too long. Local walking guidance continues."
6. Three failures in a row open the breaker for 60 s.

6. Faculty demonstration explanation

> "Online description is optional and off until the user allows it. Even then, the answer is filtered: any sentence with a distance or a claim that the path is safe is removed. Walking guidance never depends on this module."

7. Possible faculty questions

- **Why not retry a timeout?** It already used the full time; repeating it only makes the user wait longer. This fix came from a real bug where retries got a 4-second deadline that Gemini rejected.
- **Where do 30 s / 45 s come from?** They are assumptions (one live request failed at 25 s), configurable with `NEUROSONIC_VISION_TIMEOUT_S` and `NEUROSONIC_VISION_BUDGET_S`.
- **Has it worked live?** No successful live description has been recorded yet; the only approved live test timed out.

B9. Files: neurosonic/vision/gemini.py, base.py, mock.py

1. Purpose

`gemini.py` is the Google Gemini provider; `base.py` defines the provider interface, the JSON reply schema and the error types; `mock.py` is a fake provider for tests and offline demos.

2. How the code works (gemini.py)

1. `read_key()` (L26) reads `GEMINI_API_KEY` from the environment, or on Windows from the user registry environment. The key is never printed, returned or stored; error text is passed through `scrub()`.
2. `_get_client()` (L74) creates `genai.Client(api_key=…, http_options=HttpOptions(retry_options=HttpRetryOptions(attempts=1)))` – the SDK's own retries are off, so one call is one request.
3. `analyze()` (L86): `timeout_s = max(timeout_s, self.min_timeout_s)` (minimum **10 s**, because the SDK sends the timeout to Google as the server deadline and Google rejects less than 10 s); runs `generate_content` in a thread with a hard guard of timeout + 2 s; asks for JSON (`response_mime_type="application/json"`, `response_json_schema=SCENE_SCHEMA`, temperature 0.2).
4. `_map_error()` converts HTTP codes: 408/504 → `ProviderTimeout`, 429 → `ProviderRateLimited`, ≥500 → `ProviderUnavailable`, others → `Rejected`.
5. `make_provider(name)` (L159) returns a Gemini provider, a `MockProvider`, or `None` for `off`.

6. Faculty demonstration explanation

> "The key stays on the laptop. The provider sends exactly one request per attempt with a deadline of at least ten seconds, and converts every error into a clear category so the phone always has something sensible to say."

7. Possible faculty questions

- **Why the mock provider?** Tests and demos can simulate success, 503, timeouts or slow replies with no internet and no cost (`NEUROSONIC_VISION_MOCK_SCRIPT`).
- **Which model?** Configurable; default `gemini-3.8-flash` (the owner's choice).

C1. File: neurosonic/awareness.py

1. Purpose

The **live camera pipeline**. It receives one JPEG from the phone and returns the walking-path state, event messages, detections and timing. It holds the detector, runs all inference on one dedicated thread, and keeps separate state for each phone session.

2. How the code works (`process_frame`, L205)

1. Check `mode` (`chest` or `handheld`); decode the JPEG with `cv2.imdecode` (error 400 if it cannot be decoded).
2. `quality(bgr)` (L171): grey mean and standard deviation. Mean < 20 or std < 6 → `UNUSABLE` ("too dark or covered"); mean < 45 → `WARN`; otherwise `OK`.
3. If usable, `detect(bgr)` (L153) submits `_detect_on_worker` to the single-thread `ThreadPoolExecutor` and waits. The worker loads the detector on first use (`_load` → `make_detector`) and runs it.
4. `_state(session, mode)` (L189) returns the per-phone `_Session` (alert engine, `PathGuide`, `ByteTracker`, `PathEventDetector`, previous grey frame). Sessions idle for 120 s are deleted.
5. **Tracking:** `small_gray()` → `estimate_camera_motion(prev_gray, gray, track boxes)` → `st.tracker.update(dets, t, w, h, cam)`.
6. **Walking guidance:** `assess_frame(...)` gives this frame's level; `guide.update(fa, t)` confirms it over frames and says whether to announce.
7. **Events:** `st.events.update(tracks, …, path["state"])`. If an "entering" event and a generic "Caution" happen together, the more specific entering sentence replaces the generic one (L237–248).
8. **Object description** (only if `describe=True`): the WOAD `AlertEngine` (re-used) produces "Person on the left." style messages with category `description` (never interrupting).
9. Per-device counters (`self.clients`: frames, frame size, detector, processing times) are updated – counters only, no image.
10. Return a dict: `path`, `events`, `detections`, `tracks`, `camera_motion`, `detector`, `timing`, `quality`.

3. Important functions and classes

Name	Input	Processing	Output	Called by
`OBSTACLE_CLASSES` L42	—	COCO id → (name, plural, priority), 23 classes	dict	detectors, alert engine
`RGBAwareness.__init__` L80	SSDLite weights, detector preference	creates the 1-thread executor `_infer`	object	`neurosonic_app.py`
`info()` L111	—	detector name/device/precision, `recommended_frame_ms`, clients	dict	`/api/awareness/status`
`warm_up()` L144	—	detects on a black 320×240 image	True/False	`neurosonic_app.py`
`detect()` / `_detect_on_worker()` L153/L159	BGR image	runs the active detector under a lock, counts frames	detections list	`process_frame`
`quality()` L171	BGR image	brightness/contrast check	`QualityReport`	`process_frame`
`process_frame()` L205	JPEG bytes, session, mode, describe	full pipeline above	result dict	`server.py r_awareness_frame`

4. Important code sections

- **One inference thread (L91–95):** `self._infer = ThreadPoolExecutor(max_workers=1, thread_name_prefix= "neurosonic-infer")`. The comment records why: running the GPU model from ever-new request threads made memory grow about 2 MiB/s in a 10-minute soak test.
- **Frame interval (L58, L120):** `FRAME_MS_GPU, FRAME_MS_CPU = 200, 400` and `"recommended_frame_ms": FRAME_MS_GPU if det["device"] == "cuda" else FRAME_MS_CPU` – the server tells the phone how fast to send.
- **Priorities (L233):** path messages get `urgent 40 / obstacle 30 / system 20 / info 5`.

5. Connections

Called by `server.py`. Uses `detectors.py`, `tracking.py`, `pathguide.py`, `events.py`, and from the WOAD package `woad/alerts.py` (`AlertEngine`) and `woad/quality.py` (`QualityReport`, `OK/WARN/UNUSABLE`).

6. Faculty demonstration explanation

> "This is the heart of the live system. One camera frame comes in; I check the brightness, run RF-DETR on the GPU, track the objects while cancelling the phone's own movement, decide whether the walking path is clear, possibly blocked or blocked, and check whether something is moving into the path. Only changes are spoken."

7. Possible faculty questions

- **Why one inference thread?** The web server creates a new thread for every request; PyTorch/CUDA kept per-thread state that was never freed. One long-lived thread fixed the memory growth (soak test: flat memory, 0 errors).
- **Why keep state per session?** Confirmation over frames and tracking need the history of the *same* phone; two phones must not mix.
- **Does it measure distance?** No – `info()` even reports `"distance": "not available (single RGB camera)"`.
- **What if the image is too dark?** No detection runs; the path state becomes "I cannot clearly assess the path".

C2. File: neurosonic/detectors.py

1. Purpose

The two object detectors and the logic that chooses between them: **RF-DETR Nano on the NVIDIA GPU (FP16)** as the primary detector, and **SSDLite320-MobileNetV3 on the CPU** as the guaranteed fallback.

2. How the code works (`make_detector`, L161)

1. Read the preference (`auto` / `rfdetr` / `ssdlite`). For `ssdlite` return SSDLite directly.
2. Otherwise build `RFDETRDetector` (L87):
3. raise `DetectorUnavailable` if CUDA is not available, the `rfdetr` package is missing, or the local weights are not found (`find_rfdetr_weights()` looks in `models/rfdetr/` and `~/.roboflow/models/`; nothing is downloaded);
4. set `HF_HUB_OFFLINE=1`; limit PyTorch to 2 CPU threads (`GPU_MODE_CPU_THREADS`);
5. create `RFDETRNano(device="cuda", pretrain_weights=…)`;
6. try `self.model.inference(compile=False, dtype=torch.float16)` → precision `fp16`, else stay `fp32`.
7. Warm-up: detect on a black 320×240 image to prove inference works on this machine.
8. Wrap it in `FallbackDetector(primary, ssd)`. If any step fails, log the reason and return SSDLite.
9. At run time, `FallbackDetector.detect()` (L147): if RF-DETR throws an exception, switch permanently to SSDLite, restore the CPU thread count, and **re-analyse the same frame** with SSDLite.

3. Important functions and classes

Name	Input	Processing	Output	Called by
`_filter(...)` L40	raw boxes, scores, labels	keep score ≥ threshold and class in `OBSTACLE_CLASSES`	list of `{class_id, class_name, confidence, box_xyxy}`	both detectors
`SSDLiteDetector.detect()` L69	BGR image	resize to max 320 px, BGR→RGB, tensor /255, `inference_mode`	detections in original pixels	`FallbackDetector`
`RFDETRDetector.detect()` L119	BGR image	BGR→RGB PIL image, `model.predict(threshold=0.40)`	detections	`FallbackDetector`
`FallbackDetector` L125	primary, fallback factory	switch on first runtime failure	`name`, `device`, `fallback_reason`	`awareness.py`
`make_detector()` L161	classes, preference	choose, warm up, fall back	`(detector, info)`	`RGBAwareness._load`

4. Important code sections

- **CPU busy-wait fix (L30–33, L104):** with PyTorch's default 8 threads the idle threads spun between frames (measured 712 % of one core vs 121 % with 2 threads, same latency).
- **FP16 (L107–115):** the comment records the measurement: person AP@0.5 0.660 in both FP16 and FP32.

5. Connections

Used by `awareness.py`. Imports `torch`, `cv2`, `numpy`, and lazily `torchvision` (SSDLite) and `rfdetr`.

6. Faculty demonstration explanation

> "Here the detector is chosen. If CUDA, the RF-DETR package and its weights are all present, RF-DETR Nano runs on the RTX 4050 in half precision. If anything is missing – or if RF-DETR fails during a session – the app switches to SSDLite on the CPU automatically and says why. The user never loses guidance because of the GPU."

7. Possible faculty questions

- **Why RF-DETR Nano?** In our benchmark it was the most accurate (person AP@0.5 0.660 vs SSDLite 0.542), fast on the GPU (21.3 ms FP32), and Apache-2.0 licensed.
- **Why not YOLO26?** Faster, but AGPL-3.0: offering NeuroSonic over a network would oblige publishing our source.
- **What does FP16 change?** Same accuracy in our test, lower latency at 5 frames/s (53.2 vs 66.5 ms median).
- **What is the confidence threshold?** 0.40 (`score_thres`) for both detectors.
- **Was the model trained by you?** No. Both detectors use public COCO-pretrained weights; no training was done.

C3. File: neurosonic/pathguide.py

1. Purpose

Turns detections into **walking guidance**. The lower-middle part of the image is treated as the walking path, and one of five states is decided and confirmed over frames: `CLEAR_VIEW`, `POSSIBLE_OBSTACLE`, `BLOCKED_VIEW`, `UNCERTAIN`, `CAMERA_UNAVAILABLE`.

2. How the code works

1. `PATH_POLYGON` (L57) is a trapezoid in normalised image coordinates: bottom from x 0.22 to 0.78, top from 0.40 to 0.60 at y 0.45. `_path_mask()` rasterises it on a 64×48 grid.
2. `assess_frame()` (L119) for one frame:
3. unusable image → `UNCERTAIN`; sharpness (Laplacian variance) < 20 → `UNCERTAIN`;
4. for each detection with confidence ≥ 0.40, `box_path_overlap()` gives *cover* (share of the path region covered) and *inpath* (share of the box inside the region);
5. **BLOCKED** if cover ≥ 0.30 and the object is centred or mostly inside, or if inpath ≥ 0.50 and the box reaches the bottom 10 % and is at least half the image height; **POSSIBLE** if cover ≥ 0.06 or inpath ≥ 0.35;
6. a "clear" frame that is too dark (brightness < 45 or quality WARN) becomes `UNCERTAIN`.
7. `PathGuide.update(fa, t)` (L191) keeps the last 3 frames and `_decide()` (L172): `BLOCKED` if 2 of 3 frames are blocked; `POSSIBLE` if 2 of 3 are blocked or possible; `UNCERTAIN` after 2 uncertain frames in a row; `CLEAR` only after 3 clear frames; otherwise keep the current state (hysteresis).
8. Announcement policy: speak when the state changes (escalations at once; de-escalations only after 2 s of quiet), repeat "Stop … may still be blocked" every 8 s, repeat uncertain every 30 s, repeat caution only if the obstruction moved to another zone.

3. Important functions and classes

Name	Input	Output	Called by
`Thresholds` L73	—	all numeric thresholds in one dataclass	`assess_frame`
`sharpness(bgr)` L85	image	Laplacian variance of a 320 px grey copy	`awareness.py`
`box_path_overlap(box, w, h)` L93	box	(cover, inpath)	`assess_frame`, `events.py`
`assess_frame(...)` L119	detections, size, quality, brightness, sharpness	`FrameAssessment(level, reason, in_path, at_side)`	`awareness.py`
`PathGuide.update()` L191	assessment, time	dict(state, announce, text, category)	`awareness.py`

6. Faculty demonstration explanation

> "A blind user does not need a list of object names every half second. This file decides one calm state for the walking path and speaks only when it changes. 'Clear' needs three good frames, warnings need two of three, and a dark or blurred image can never be called clear. The message is 'appears clear, continue carefully' – never 'safe'."

7. Possible faculty questions

- **Why a trapezoid?** A chest-mounted camera sees the walking path widening towards the bottom of the image, like a corridor in perspective.
- **Does lower in the image mean closer?** Not reliably, so the code never claims a distance.
- **Are the thresholds validated?** No. They are engineering defaults, not tested with users or on Vellore streets.
- **Why hysteresis?** Without it the state would flicker between clear and caution on noisy detections.

C4. File: neurosonic/tracking.py

1. Purpose

Follows each object from frame to frame and measures its motion **relative to the background**, so that the user's own walking or turning does not look like objects moving.

2. How the code works

1. `estimate_camera_motion(prev_gray, gray, exclude_boxes)` (L41): find up to 150 corner points (`cv2.goodFeaturesToTrack`) outside the detected objects, follow them with Lucas–Kanade optical flow (`cv2.calcOpticalFlowPyrLK`), and fit a similarity transform (shift, scale, rotation) with RANSAC (`cv2.estimateAffinePartial2D`). At least 12 inlier points are needed, otherwise `ok=False`.
2. `ByteTracker.update(dets, t, w, h, cam)` (L125), ByteTrack-style:
3. move each track's last box by the camera motion (where it would be if it stood still);
4. match **confident** detections (≥ 0.55) by IoU ≥ 0.25, then **lower-confidence** ones (0.40–0.55) to the remaining tracks with IoU ≥ 0.45, always within the same class;
5. for matched tracks store `rel_dx, rel_dy` = observed centre minus predicted static centre (image fractions);
6. delete tracks missed more than 3 frames; start new tracks only from confident detections.

6. Faculty demonstration explanation

> "When the person carrying the phone walks, everything in the picture moves. Here I estimate that global movement from the background and subtract it, so a parked car that slides across the image while the user turns is not treated as moving. The tracker is a small implementation of the ByteTrack idea, without an extra library."

7. Possible faculty questions

- **What is IoU?** Intersection over union: overlap area divided by combined area of two boxes (0 to 1).
- **Why two matching rounds?** Low-confidence detections help keep an existing track alive but should not start new ones (fewer false tracks).
- **Is this the official ByteTrack code?** No, a small own implementation of the same idea (cited in the docstring).

C5. File: neurosonic/events.py

1. Purpose

Conservative **event alerts** from the tracks: an object **entering** the walking path from a side, or **crossing** in front of the user. "Approaching" is deliberately not announced.

2. How the code works (`PathEventDetector.update`, L59)

1. Keep a short history of camera motion; return nothing if the state is `UNCERTAIN`, `CAMERA_UNAVAILABLE` or `BLOCKED_VIEW`, or if an event was spoken less than 3 s ago (`GLOBAL_GAP_S`).
2. Consider only confirmed tracks (≥ 3 hits, ≥ 0.4 s old) of classes that move by themselves (`MOVERS`: person, bicycle, motorcycle, car, bus, truck, dog, cat, cow, horse).
3. Require a **stable camera** in the window (shift ≤ 6 % of the width per frame, scale change ≤ 4 %).
4. `_entering()` (L87): within 1.5 s the box was outside the path region and now overlaps it, and its relative horizontal speed towards the centre is ≥ 0.12 image widths per second → "Caution. A bicycle is moving into your path from the left."
5. `_crossing()` (L112): within 3 s the centre moved from < 0.35 to > 0.65 of the width (or back), speed ≥ 0.15, box in the lower half → "A person is crossing in front of you, from left to right."
6. Each event type is spoken once per track, at most one event per frame.

6. Faculty demonstration explanation

> "Objects alone are not enough – what matters is movement into the path. Using motion relative to the background, these rules detect someone stepping into the path or crossing in front. I deliberately left out 'approaching', because with a walking camera every object grows in the picture, which would cause false alarms."

7. Possible faculty questions

- **How fast is the entering alert?** On the demo clip about 1 s after the rider first appears.
- **Why suppress events while the camera shakes?** Object motion cannot be separated from the user's motion then.
- **Why no events during "Stop"?** The urgent "Stop" message is already the right warning.

D1. File: woad/model_loader.py

1. Purpose

Loads the base-paper **WOAD exp72** checkpoint safely: it verifies the file's SHA-256 fingerprint, loads the pickled PyTorch model without importing the broken original training code, and checks the architecture.

2. How the code works (`load_model`, L76)

1. Check the file exists; compute `sha256()` (L34). For the default `weights/best.pt`, require `EXP72_BEST_SHA256 = "da043ff2e1edd393…"` (L24); otherwise raise `CheckpointError` ("modified or replaced").
2. Inside `_legacy_module_aliases()` (L46), temporarily map the module names `models`, `models.yolo`, `models.common` (used inside the pickle) to our copied classes in `woad.arch`, then restore `sys.modules`.
3. `torch.load(weights, map_location="cpu", weights_only=False)` – required because the YOLOv5-format checkpoint stores a whole `nn.Module`.
4. Take `ckpt["ema"]` or `ckpt["model"]`; check it is `arch_yolo.Model` and that its children equal `EXPECTED_CHILDREN` (refuses the different 5-class root `best.pt`).
5. `model.float().eval().to(device)`; freeze all parameters; return `(model, info)` with names, number of classes, strides, parameter count (7,521,992) and device.

6. Faculty demonstration explanation

> "The original checkpoint can only be loaded with the authors' code, which no longer works with current PyTorch and NumPy. Here I load it with a copy of only the nine classes it needs, after checking its fingerprint, so a wrong or modified file is refused."

7. Possible faculty questions

- **Is `weights_only=False` dangerous?** Unpickling can run code, so only the hash-verified, trusted file is loaded.
- **How do you know the copy is identical?** `tools/verify_equivalence.py` runs both versions on the same input; the maximum difference is 0.0.

D2. Files: woad/arch/yolo.py and woad/arch/common.py

1. Purpose

The WOAD neural network (copied from the original repository): a YOLOv5-style detector with **two input streams** – a near-infrared image and a colour-mapped depth image – fused by a transformer.

2. How the code works (`Model._forward_once`, yolo.py L165)

1. The depth stream (`self.model1`: Conv, Conv, C3) produces depth features.
2. The IR stream runs through `self.model` layer by layer. After layer 2 (`if i == 2`, L175) the fusion happens:
3. both feature maps are weighted 0.5 / 0.5 (`init_score`);
4. average-pooled to 8×8 (`self.avp`), flattened and concatenated as tokens, plus a position embedding;
5. passed through the transformer blocks (`self.trans_blocks`, 8 blocks of self-attention) and normalised;
6. split back into IR and depth parts, upsampled, and multiplied with each stream's convolution features;
7. concatenated and reduced with `conv3` → continue through the rest of the YOLO network.
8. The `Detect` head outputs 25,200 candidate boxes per image (8 values each: box, objectness, 3 class scores).

`common.py` holds the building blocks `Conv`, `C3`, `SPPF`, `Concat`. `Model.__init__` deliberately raises an error: the model can only come from the checkpoint (training is not part of this package).

6. Faculty demonstration explanation

> "This is the base paper's model. The infrared and depth images go through two branches, and after the second layer a transformer lets every region of one image attend to every region of the other. That is the 'cross-modal fusion'. We measured that IR matters most: blanking the IR removes all detections, blanking depth keeps about 65 %."

7. Possible faculty questions

- **Why can't the phone camera use WOAD?** It needs the CS30 sensor's IR + colour-depth pair; an RGB photo is a different kind of input.
- **How good is it?** The authors report mAP@0.5 0.669 from their plot (not verifiable). On the published dataset we measured person AP@0.5 = 0.218.
- **Classes?** person, Ebike, Car.

D3. Files: woad/preprocess.py and woad/ops.py

1. Purpose

Prepare inputs exactly as during training, and provide the YOLO helper functions: letterbox, non-maximum suppression (NMS) and mapping boxes back to the original image.

2. How the code works

1. `read_image()` (preprocess L36) reads 8-bit images; a grey or 16-bit depth image is rejected.
2. `preprocess_ir()` (L62): resize the long side to 640 (INTER_AREA when shrinking), letterbox to 640×640 with grey padding 114. `preprocess_depth()` (L72): `cv2.medianBlur(depth, 3)`, then letterbox.
3. `_to_tensor()`: BGR→RGB, HWC→CHW, divide by 255. `prepare_pair()` (L87) returns both tensors plus originals.
4. `non_max_suppression()` (ops L74): drop boxes below the confidence threshold and keep the best box among overlapping boxes of the same class (IoU > 0.45). `scale_coords()` (L161) undoes the letterbox.

6. Faculty demonstration explanation

> "A model only works if the input is prepared exactly as in training. I copied these rules from the original validation code, and mutation tests prove it: removing the median blur or the colour conversion makes tests fail."

7. Possible faculty questions

- **What is letterboxing?** Resizing while keeping the aspect ratio and padding the rest, so the image becomes square.
- **What is NMS?** Non-maximum suppression: among overlapping boxes for the same object, keep only the highest score.

D4. File: woad/detector.py

1. Purpose

`WOADDetector`: one simple interface around loading, preprocessing, the forward pass, NMS and box mapping.

2. How the code works

`__init__` (L20) calls `load_model`. `raw_forward(ir, depth)` (L45) runs `self.model(ir, depth)[0]` under `torch.inference_mode()` → shape (B, 25200, 8). `detect_prepared()` (L49) applies NMS (`conf_thres` 0.25 default, `iou_thres` 0.45), scales boxes to original IR pixels and returns `detections` plus timings. `detect_arrays()` and `detect_files()` accept images or file paths.

6. Faculty demonstration explanation

> "This is the WOAD equivalent of a detector class: two images in, a list of boxes with class names and confidences out, in the original image's pixel coordinates."

7. Possible faculty questions

- **Who uses it?** `neurosonic/service.py` (replay and upload), `woad_detect.py` and `tools/evaluate.py`.

D5. File: woad/quality.py

1. Purpose

The **input-quality gate** for WOAD frames: it decides whether a frame is similar enough to the training data for the detections to mean anything. Unusable frames never reach the detector.

2. How the code works (`InputQualityChecker.check`, L55)

1. Compute the IR grey **mean** and **standard deviation** (contrast) and the share of **invalid (black) depth pixels**.
2. Apply `QualityThresholds` (L28): mean < 40 or std < 10 or > 95 % invalid depth → `UNUSABLE`; mean < 100 or std < 40 → `WARN`; different IR/depth sizes → `WARN`; otherwise `OK`.
3. Return `QualityReport(status, reasons, ir_mean, ir_std, depth_invalid)`.

The thresholds were calibrated on our data (docstring table): 100 % of `detec_test` passes; the dark CS30 recording is 42.5 % unusable and 56.1 % warn.

6. Faculty demonstration explanation

> "We found the model is blind on dark infrared images – the sensor recording averaged brightness 59 versus 135–142 in training-like data. Instead of guessing, this gate refuses such frames and the system says 'Obstacle detection unavailable: IR image too dark'. A silent false 'all clear' would be worse."

7. Possible faculty questions

- **Why not brighten dark images?** We tested five corrections; the best one found one real person but produced 56 false alarms, so none was adopted.
- **Is the camera pipeline using the same gate?** It uses the same `QualityReport` type, with its own thresholds for phone photos (`awareness.py quality()`).

D6. File: woad/alerts.py

1. Purpose

`AlertEngine` turns flickering detections into a few stable, non-repetitive spoken messages, and reports when detection is unavailable. Used by the WOAD replay/upload and by Object Description mode for the phone camera.

2. How the code works (`process`, L140)

1. If the input is unusable or missing for longer than `unavailable_after_s`, say "Obstacle detection unavailable: "; after recovery, "resumed".
2. Keep detections with confidence ≥ `alert_conf` (0.40).
3. Match them to existing tracks by IoU ≥ 0.30 (per class); each track remembers hits in the last 3 frames.
4. A track is **confirmed** with 2 hits in 3 frames (`confirm_hits`, `confirm_window`).
5. Position = left / ahead / right by thirds of the image.
6. Announce once; repeat only after 15 s (`repeat_s`) or a position change; merge similar objects ("2 people detected ahead"); at most 2 messages per frame, ordered by class priority.

`AlertConfig` (L31) holds all thresholds and wording (the camera uses chest/handheld wording and no "detected" word).

6. Faculty demonstration explanation

> "A detector produces many boxes per second; a user needs a few correct sentences. This engine confirms each object over 2 of 3 frames, says where it is, and avoids repeating itself."

7. Possible faculty questions

- **Why 2 of 3?** One-frame false positives are not spoken; it costs about one frame of delay. The values are engineering defaults, not user-validated.
- **Are messages invented?** No; they are fixed templates filled only from detections in the current frame (a randomised test checks this).

D7. Files: woad/sources.py, replay.py, pairing.py, visualize.py, metrics.py, woad_detect.py, woad_replay.py

1. Purpose

Support code and command-line tools for the base-paper reproduction.

2. What each does

File	Role
`woad/pairing.py`	`pair_directories()` pairs IR and depth files by exact filename and lists unpaired files
`woad/sources.py`	`ReplaySource` yields `Frame` objects with synthetic timestamps (index / fps) and detects clip changes from filename-number gaps
`woad/replay.py`	`run_replay()`: the offline replay pipeline (quality → detector or cached detections → alert engine)
`woad/visualize.py`	draws boxes and makes the IR/depth side-by-side image
`woad/metrics.py`	the original YOLOv5 scoring code (`ap_per_class`, `process_batch`), proven identical by a test
`woad_detect.py`	CLI: detect on one pair, two folders or a `--source` folder; writes annotated images, JSON and `results.json` in a new run folder (never overwrites)
`woad_replay.py`	CLI: replays recorded frames through detector + alert engine, writes `transcript.txt` / `.jsonl`; two runs give byte-identical transcripts

6. Faculty demonstration explanation

> "With these tools I reproduced the base paper on Windows: `woad_detect.py --source demo_data/detec_test` processes the authors' 600 recorded frames, and `woad_replay.py` shows which spoken alerts the system would produce."

7. Possible faculty questions

- **What does `detec_test` show?** Detections in 313 of 600 frames (person 271, Ebike 160, Car 52), with some visible errors (a missed person, cars on shrubs).

E1. File: web/index.html

1. Purpose

The single web page of the app. It defines all screens, buttons, status badges and the spoken-alert banner, and loads `style.css` and the JavaScript module `app.js`.

2. How it is organised

- `<meta name="viewport" content="…, viewport-fit=cover">` lets the page use the whole iPhone screen (notch area).
- **Header:** status badges `b-gps`, `b-voice`, `b-det` (laptop server), `b-net` (internet).
- **Alert banner** `#alert` with `role="status" aria-live="assertive"` – screen readers (VoiceOver) read new alerts.
- **Screens** (`<section class="tab">`): `tab-detect` (Home: camera, path-state card, Start/Stop, detector chip, Object Description switch, Describe surroundings), `tab-nav` (Navigate: map, search, route, guidance, location panel), `tab-hist` (History), `tab-set` (Settings: Test Voice, carrying mode, consent, permissions, offline status), `tab-how` (How it works), `tab-sim` (Research tools: WOAD replay, image-pair upload, event-camera simulation).
- **Bottom bar** `<nav class="tabs">`: Home, Navigate, the central **Speak** button, History, Settings, How it works.

6. Faculty demonstration explanation

> "The whole interface is one HTML page with six screens. Only one is visible at a time; `app.js` switches them. The alert banner is an ARIA live region, so VoiceOver can read every alert too."

7. Possible faculty questions

- **Why a web app and not a native iOS app?** No App Store build was needed, it works in Safari, and all AI runs on the laptop. A native app (with LiDAR) is future work.
- **Is it accessible?** Large buttons (min 48 px height), high contrast, ARIA labels and a live region. Not yet tested with VoiceOver users.

E2. File: web/style.css

1. Purpose

Mobile-first, high-contrast dark theme for the iPhone.

2. Key parts

- `:root` colour variables (`--bg #0b1020`, `--ok` green, `--warn` orange, `--bad` red, `--accent` yellow).
- Safe-area variables `env(safe-area-inset-*)` keep content clear of the notch and home indicator.
- Touch targets: `button { min-height: 48px }`, map controls 44 px.
- Path-state card colours: `.pathstate.clear` green, `.caution` orange, `.stop` red background, `.unavailable` dashed.
- `@media (min-width: 900px)` for the laptop browser; `prefers-reduced-motion` respected.

6. Faculty demonstration explanation

> "The design is for a phone held in one hand or worn on the chest: big buttons, strong colours that match the spoken state – green, orange, red – and nothing hidden under the iPhone's notch."

E3. File: web/js/app.js

1. Purpose

The main frontend program (1,015 lines). It connects the screens, the phone's sensors (camera, microphone, GPS), the laptop API, the speech queue and navigation into one voice-first flow.

2. How the code works

1. Imports `Announcer` (speech.js), navigation maths (nav.js), `MapView`, `recordUtterance`, event-simulation code.
2. Loads settings from `localStorage` with `DEFAULTS` (L13): neural voice, chest mode, `describe: false`, `cloudConsent: false`.
3. `api(path, body)` (L32): a small `fetch` wrapper – GET without body, POST with JSON; throws on HTTP errors.
4. `init()` (L66) calls `/api/status`; `pollEvents()` (L88) asks `/api/events?since=` every 1 s (3 s when hidden) and announces WOAD replay events; it also says "Connection to the laptop lost / restored".
5. Status checks on start-up: `/api/awareness/status` (L812), `/api/tts/status`, `/api/map/info`, `/api/vision/status`.
6. **Camera awareness:** `startAwareness()` (L541) → `scheduleFrame()` → `awarenessStep()` (L580) repeatedly (see Part 9 for line-by-line). `drawAwareness()` (L638) draws the path trapezoid and boxes on an overlay canvas.
7. **Voice commands:** `listenOnce()` (L817) records and posts WAV to `/api/voicecmd`; `runIntent()` (L842) acts on the intent (navigate → choose destination and plan route, then wait for "start navigation"; status; repeat; …).
8. **Navigation:** `runSearch()` (`/api/map/search`), `planRoute()` (L353, `/api/route`), `onPosition()` (L219) for each GPS fix, `followActiveRoute()` (L402) speaks instructions and re-routes when off route.
9. **History:** `loadHistory()` (L486) loads `/api/stats` and `/api/history?limit=100`.
10. **Online description:** `describeScene(question)` (L737) sends one photo only if consent was given.

3. Important functions

Function	What it does	Talks to
`api()` L32	JSON fetch helper	every JSON endpoint
`showTab()` L43	switch screens, load data for History/Navigate/Settings	DOM
`startAwareness()` / `stopAwareness()` L541/L567	open/close rear camera, start/stop frame loop	`getUserMedia`
`awarenessStep()` L580	capture → JPEG → POST → speak results	`/api/awareness/frame`
`renderDetector()` L518	the "RF-DETR · GPU" / "SSDLite · CPU" chip	response `detector`
`listenOnce()`, `runIntent()` L817/L842	voice command round trip	`/api/voicecmd`
`planRoute()`, `followActiveRoute()` L353/L402	route planning and following	`/api/route`, nav.js
`describeScene()` L737	opt-in cloud description	`/api/vision/describe`

6. Faculty demonstration explanation

> "This file is the glue on the phone. When I press Start it opens the rear camera, and every 200 milliseconds it sends one small JPEG to the laptop, never more than one at a time. The answer contains the path state and any sentences to speak, which go to the speech queue. The same file handles voice commands, the map and history."

7. Possible faculty questions

- **Why polling for events instead of WebSockets?** The standard-library server does not support WebSockets easily; polling once per second is tiny traffic and survives reconnects. Camera results come back directly in each reply.
- **What if the laptop does not answer?** Each frame request is cancelled after 5 s (`AbortController`); after 3 failures the app says "Camera view is unavailable".
- **Why 320 px frames?** Small enough for fast upload over Wi-Fi; RF-DETR Nano works at 384 px input anyway.
- **Does the phone store images?** No; each frame is drawn to an off-screen canvas, sent, and replaced.

E4. File: web/js/speech.js

1. Purpose

The speech system: a **priority queue** so the most important message is spoken first, nothing overlaps, outdated warnings are dropped, and urgent warnings interrupt routine speech. Plays the Piper voice, with the browser voice as fallback.

2. How the code works

1. `PRIORITY` (L4): urgent 4 > system 3 > obstacle 2 > motion/navigation/scene 1 > info/description 0.
2. `SpeechQueue.push(text, category, now, key)` (L34): skip a sentence said within its `COOLDOWN_MS`; a `key` (e.g. `"path"`) makes newer path messages replace older queued ones ("latest wins"); keep at most 6 items sorted by priority.
3. `next(now)` (L47) drops items older than `MAX_AGE_MS` (an obstacle warning older than 4 s is not spoken).
4. `Announcer.say()` (L156) pushes the message, plays a short beep (panned left/right), and if a more important message arrives while speaking, `interrupt()` stops the current one; an interrupted navigation message is re-queued once (`requeue`, L68).
5. `_pump()` (L174) plays the next item: `_playNeural()` fetches `/api/tts?text=` and plays it through one `<audio>` element; on failure `_playBrowser()` uses `speechSynthesis`. A 250 ms gap (`GAP_MS`) separates messages.
6. `unlock()` (L118) must run inside the first tap – iOS only allows audio started by a user gesture.
7. `prefetch()` (L211) downloads the fixed warning audio in advance.

6. Faculty demonstration explanation

> "Speech is the user's only output, so it must be prioritised. 'Stop' interrupts a navigation instruction, which is then repeated once. Old warnings are dropped because a four-second-old obstacle warning is misleading."

7. Possible faculty questions

- **What happens if Piper is unavailable?** The phone's own voice is used automatically.
- **Why does the user have to tap once?** iOS blocks audio that was not started by a tap; `unlock()` plays a silent clip during that tap.

E5. File: web/js/nav.js

1. Purpose

Pure navigation maths (no screen code), so it can be unit-tested in Node.js.

2. Key functions

- `distanceM()` (haversine distance), `bearingDeg()`, `compass()`, `formatDistance()`.
- `assessFix(fix, now)` with `GPS_LIMITS = {staleMs: 15000, poorM: 50, unusableM: 100}`: fixes older than 15 s or worse than 100 m are not used.
- `gpsWatch()`: after 20 s without a good fix during guidance → "GPS signal lost. Guidance paused…".
- `projectOnPolyline()`, `prepareRoute()`, `followRoute()` (L116): pre-announce a turn about 70 m ahead, say it at 25 m, declare "off route" after 2 fixes more than 35 m (or 1.5× accuracy) away, "arrived" within 15 m.
- `canReroute()`: at most one re-route every 15 s.

6. Faculty demonstration explanation

> "Route following projects each GPS position onto the route line, then decides whether to pre-announce a turn, say it now, report arrival, or recalculate because the user left the route."

E6. Files: web/js/mapview.js, recorder.js, eventsim.js

1. Purpose and how they work

- **mapview.js** – `MapView` draws the offline `map.json` on a `<canvas>` with an equirectangular projection (x = Δlon·111,320·cos(lat), y = −Δlat·110,540 metres), supports drag/pinch/zoom, and draws the route, destination, position, scale bar and OpenStreetMap attribution. It redraws sharply for the device pixel ratio.
- **recorder.js** – `recordUtterance()` records from the microphone, `EndOfSpeech` stops after about 1 s of quiet (or 6 s maximum), `downsample()` converts to 16 kHz and `encodeWav()` builds a 16-bit mono WAV for Vosk.
- **eventsim.js** – a *demonstration only* "event camera" simulation: `diffEvents()` marks pixels whose brightness changed by more than a threshold, and `MotionAlerter` says "Motion on the left/ahead/right". It is labelled in the UI as not WOAD and not obstacle detection, and lives under Research tools.

6. Faculty demonstration explanation

> "The map is drawn by our own small renderer, so it works offline. The recorder produces exactly the audio format the offline recogniser needs. The event-camera simulation is only a demonstration of the idea."

7. Possible faculty questions

- **Why not Google Maps or Leaflet tiles?** They need internet; our own canvas renderer uses the local map file.
- **Why 16 kHz?** The Vosk model is trained on 16 kHz audio.

F1. File: tools/benchmark_detectors.py

1. Purpose

The **Phase 1 controlled benchmark** that chose the detector: SSDLite vs RF-DETR Nano vs YOLO26n/s, each on CPU and GPU, with identical inputs. Not used by the running app. Output: `outputs/benchmarks/phase1/results.json`.

2. How it works

- **Quality:** the published WOAD dataset `VOCdevkit_nc_10` validation split (365 near-IR images, 129 people); person AP@0.5 and AP@0.5:0.95 computed with the project's own metric code (`woad.metrics.ap_per_class`).
- **Speed:** the first frames of the recorded `detec_test` sequence (640×480); latency = preprocessing + inference + postprocessing on an in-memory frame; also CPU load and GPU memory.

6. Faculty demonstration explanation

> "Before replacing the detector I measured all candidates on the same images. RF-DETR Nano was the most accurate and fast enough on the GPU, so it became the main detector, with SSDLite kept as the CPU fallback."

7. Possible faculty questions

- **Are these phone images?** No – near-infrared images, the only labelled data available. The comparison between detectors is fair; absolute numbers for phone photos are unknown.

F2. File: tools/benchmark_precision.py

1. Purpose

Measures RF-DETR Nano **FP32 vs FP16**: accuracy on the same 365 images, and latency at the phone frame size (320×240), both back-to-back and **paced at 5 frames/s** (as in the app).

2. Result (`outputs/benchmarks/precision/results.json`)

	FP32	FP16
Person AP@0.5	0.6597	0.6600
Recall at 0.40	0.7287	0.7287
Latency back-to-back (median)	17.3 ms	16.9 ms
Latency paced at 5 frames/s (median / p90)	66.5 / 78.6 ms	53.2 / 60.8 ms

6. Faculty demonstration explanation

> "Pacing matters: at five frames per second the laptop GPU drops to a low-power state between frames (we logged P8 at about 200–400 MHz), so real latency is higher than a back-to-back benchmark. FP16 keeps the same accuracy and cuts the paced latency from 66.5 to 53.2 milliseconds."

F3. File: tools/soak_test.py

1. Purpose

A long-duration stability test of the running server: streams the demo clip to `/api/awareness/frame` at 5 frames/s (one request at a time, like the phone) and records every 10 s: frames/s, latency, server memory (RSS), GPU memory and utilisation, events. Optional slow mock cloud requests run in parallel.

2. Results (`outputs/benchmarks/soak/`)

- **Before the fix:** server memory grew about 2 MiB per second; request medians 62–104 ms.
- **After the fix (one inference thread):** 2,980 frames in 10 minutes, **0 errors**, 4.97 frames/s, medians 70–78 ms, no upward memory trend, 7 of 7 slow (20 s) mock cloud requests answered.

7. Possible faculty question

- **Does 10 minutes prove there is no leak?** No; it rules out the large leak we found. A run of an hour or more is still pending.

F4. Files: tools/evaluate.py and tools/verify_equivalence.py

1. Purpose

- `evaluate.py` evaluates the WOAD checkpoint on a labelled IR + depth dataset, following the original validation pipeline (conf 0.001, IoU 0.6, max 300 boxes, IoU 0.50:0.05:0.95, original `ap_per_class`). A `--class-map` option maps dataset classes to model classes.
- `verify_equivalence.py` runs the original model code in a separate process and our copy on the same seeded random inputs and compares the outputs (exit code 0 = identical).

2. Results

Maximum difference 0.0. Person AP@0.5 on the published dataset: **0.218** (val, 365 images) and 0.173 (train, 1,286 images). The authors' reported 0.669 mAP could not be verified, because their exact training data is not published.

6. Faculty demonstration explanation

> "Reproducing a paper means proving the code is the same and measuring honestly. Our copy gives exactly the same output as the original. On the data that is actually published, the model finds people with AP 0.218 – much lower than the authors' plot, mostly because it misses people in new scenes."

F5. Folder: tests/ (Python, Node.js and browser tests)

1. Purpose

Automated proof that each part works and keeps working after changes.

2. What is in it

Test file(s)	What they check
`test_model.py`, `test_detection.py`, `test_preprocess_pairing.py`, `test_cli.py`, `test_evaluate.py`, `test_demo_detec_test.py`	WOAD loading and hash, two-input forward pass, preprocessing, pairing, CLI outputs, golden detections, metric equivalence
`test_alerts.py`	alert engine: confirmation, repetition, merging, unavailable/resumed
`test_app.py`, `test_mobile_access.py`	real server on a test port: every API route, errors, path-traversal blocking, TLS certificate rules, the "one idle client froze the server" regression
`test_voice_and_map.py`	Piper, Vosk intents, place matching, A* routes vs an independent shortest-path check
`test_pathguide.py`, `test_refinement_ui_nav.py`	path states, thresholds, route steps, interface structure
`test_perception_pipeline.py`	detector fallback, tracking, camera-motion compensation, entering/crossing events (RF-DETR GPU tests are skipped in the CPU environment)
`test_vision.py`, `test_vision_live.py`	online description with mock and fake clients (timeouts, 503, breaker, filter); the live test runs only with `NEUROSONIC_LIVE_VISION=1`
`test_final_integration.py`	offline voice commands, live camera awareness, their APIs, and offline operation
`test_web_js.py + js/*.test.mjs`	runs the JavaScript unit tests in Node.js (speech queue, navigation maths, GPS states, event simulation)
`js/ui_*_cdp.mjs` (7 files)	browser tests: headless Chrome with a fake camera and microphone, at iPhone size, checking speech, guidance, events, vision consent, no external requests and no JavaScript errors

`_offline.py` is a helper used in several tests to forbid any network access except to the laptop itself. Those tests show that voice, map search and routing work without internet in-process; it does not replace a test with the network physically disconnected (still pending).

Final result (3 Nov 2026): `Ran 204 tests … OK` in `.venv-gpu` (1 skipped: the opt-in live Gemini test) and in `.venv` (3 skipped: GPU-only tests); browser tests **7 of 7 pass**.

Command: `..\.venv-gpu\Scripts\python -m unittest discover -s tests -t .`

6. Faculty demonstration explanation

> "There are 204 automated tests. Some start a real server and call every endpoint, some run the JavaScript logic in Node, and seven drive a real Chrome browser with a fake camera. Several tests block all non-local network access, which shows the voice, map and routing work without internet. None of this is a substitute for the iPhone and street tests, which are still pending."

Part 4 – AI Engine and Model Explanation

NeuroSonic contains **three** neural networks. Only the first two run on the live phone camera.

Model	Role in the app	Framework	Device	Weights file	Licence
RF-DETR Nano	Primary live camera detector	PyTorch + `rfdetr` 1.11.2	NVIDIA RTX 4050 (CUDA), FP16	`models/rfdetr/rf-detr-nano.pth`	Apache-2.0
SSDLite320-MobileNetV3-Large	Automatic fallback detector	PyTorch + torchvision	CPU, FP32	`models/rgb/ssdlite320_mobilenet_v3_large_coco-a79551df.pth`	BSD-3
WOAD exp72 (base paper)	Research mode: replay and IR + depth upload	PyTorch (own copy of the original code in `woad/arch/`)	CPU (`--device cpu`)	`weights/best.pt` (SHA-256 checked)	from the original repository

Both live detectors use **public COCO-pretrained weights**. No model was trained or fine-tuned in this project. **TensorFlow is not used anywhere.**

4.1 RF-DETR Nano (primary detector)

What it is. RF-DETR is a *detection transformer* (DETR family) from Roboflow. A pre-trained vision-transformer backbone turns the image into feature tokens; a transformer decoder then uses a fixed set of learned "object queries", and each query predicts at most one object (a box plus class scores). Because each object is predicted once, DETR-style models do **not** need non-maximum suppression. The Nano version works at **384 × 384 pixels** (`input_size = 384`).

How NeuroSonic loads it (`detectors.py` `RFDETRDetector.__init__`, L90):

1. Refuse to start (raise `DetectorUnavailable`) if `torch.cuda.is_available()` is false, the `rfdetr` package is missing, or the weights are not on disk. **Nothing is downloaded at run time** (`HF_HUB_OFFLINE=1`).
2. Limit PyTorch's CPU threads to 2 (`GPU_MODE_CPU_THREADS`), because the GPU does the heavy work.
3. `RFDETRNano(device="cuda", pretrain_weights=...)` puts the network on the GPU.
4. `self.model.inference(compile=False, dtype=torch.float16)` switches it to **FP16**. If that fails, it stays FP32.

How one frame is detected (`RFDETRDetector.detect`, L119): BGR image → RGB (OpenCV stores colours as BGR) → PIL image → `model.predict(image, threshold=0.40)` → boxes `xyxy`, `confidence`, `class_id` → `_filter()` keeps only the 23 obstacle classes.

4.2 SSDLite320-MobileNetV3 (fallback detector)

What it is. A *single-shot detector*: one pass of a light convolutional network (MobileNetV3-Large backbone) predicts scores and box corrections for many fixed "anchor" boxes at once; torchvision then removes duplicate boxes with NMS inside the model. It was the project's original live detector and is designed for phones and CPUs.

How one frame is detected (`SSDLiteDetector.detect`, L69):

1. Shrink the image so its longest side is at most 320 px (`cv2.INTER_AREA`); remember the `scale`.
2. BGR → RGB, `HWC → CHW` (`permute(2, 0, 1)`), convert to float and divide by 255 (pixel values 0–1).
3. Run the model inside `torch.inference_mode()` (no gradient bookkeeping, less memory, faster).
4. `_filter(..., scale)` keeps score ≥ 0.40 and obstacle classes, and divides boxes by `scale` so they are in the original image's pixels.

4.3 Choosing and switching detectors (GPU first, CPU fallback)

```
neurosonic_app.py --detector auto (default)
      │
      ▼
make_detector()  ──► RF-DETR possible? (CUDA ✓, rfdetr ✓, weights ✓, FP16 warm-up on a black 320×240
image ✓)
      │                          │ yes                                  │ no (any step fails)
      ▼                          ▼                                      ▼
FallbackDetector(primary = RF-DETR on GPU)              FallbackDetector(primary = SSDLite on CPU)
      │                                                  + fallback_reason shown in
/api/awareness/status
      ▼
during the session: RF-DETR raises an exception  ──►  switch to SSDLite permanently, restore CPU
threads,
                                                      re-analyse the SAME frame (user gets an answer
anyway)
```

The phone is told the right speed: `recommended_frame_ms` is **200 ms on the GPU** and **400 ms on the CPU** (`awareness.py` L58, L120).

4.4 Which objects are reported

`OBSTACLE_CLASSES` (`awareness.py` L42) maps 23 COCO category ids to a name, a plural and a priority – for example person, bicycle, car, motorcycle, bus, truck, dog, cow, bench, chair, potted plant, fire hydrant, stop sign. Everything else (cups, laptops, food …) is ignored.

Limits that must be stated honestly: COCO has no classes for steps, kerbs, potholes, poles or walls, so neither detector can see them. A single RGB camera gives **no distance** (`info()` reports `"distance": "not available (single RGB camera)"`).

4.5 Thresholds and post-processing in one table

Stage	Value	Where
Detector confidence	≥ 0.40	`score_thres` in `detectors.py`, `awareness.py`
Image unusable	grey mean < 20 or std < 6	`awareness.py quality()`
Image dark (warning)	mean < 45	`awareness.py quality()`
Image blurred → "uncertain"	Laplacian variance < 20	`pathguide.py`
Path BLOCKED (one frame)	covers ≥ 30 % of path region and centred/mostly inside, or large box at the bottom	`pathguide.py assess_frame`

Stage	Value	Where
Path POSSIBLE obstacle	covers ≥ 6 % or ≥ 35 % of box inside region	`pathguide.py`
Confirmation over frames	BLOCKED 2 of 3; CLEAR needs 3 clear frames	`PathGuide._decide`
Tracker matching	IoU ≥ 0.25 (confident ≥ 0.55), IoU ≥ 0.45 (0.40–0.55)	`tracking.py`
Camera-motion estimate	≥ 12 RANSAC inlier points	`tracking.py`
Entering event	relative speed ≥ 0.12 widths/s, within 1.5 s	`events.py`
Crossing event	0.35 → 0.65 of the width within 3 s, speed ≥ 0.15	`events.py`
WOAD detections	conf 0.25, NMS IoU 0.45	`woad/detector.py`

4.6 CUDA, FP16 and CPU fallback in simple words

- **CUDA** lets PyTorch send the network's matrix calculations to the thousands of small cores on the NVIDIA GPU.
- **FP16** stores numbers in 16 bits instead of 32. The GPU moves half as much data and uses its fast half-precision units. Measured result (`benchmark_precision.py`): person AP@0.5 **0.6600 (FP16) vs 0.6597 (FP32)**, paced median latency **53.2 ms vs 66.5 ms**.
- **CPU fallback** means the app never depends on the GPU: if CUDA is missing (for example started from `.venv`), SSDLite runs on the CPU, and the phone sends frames every 400 ms instead of 200 ms.

4.7 The base-paper WOAD model (research mode)

WOAD is a YOLOv5-style detector with **two inputs**: a near-infrared image and a colour-mapped depth image from the CS30 sensor. After layer 2 a **transformer fuses the two streams** (8 attention blocks on 8×8 pooled tokens). The head outputs 25,200 candidate boxes; NMS keeps the best ones. Classes: person, Ebike, Car.

It is **not** used on the phone camera, because an ordinary RGB photo is a different kind of input. In NeuroSonic it powers the replay screen (`/api/replay/*`) and the IR + depth upload (`/api/detect`). Measured on the published data: person AP@0.5 **0.218**; the authors' reported 0.669 mAP could not be verified.

4.8 Optional online scene description (not a detector)

`neurosonic/vision/` sends **one photo, only after the user agrees**, to Google Gemini (default model `gemini-3.8-flash`, key read from `GEMINI_API_KEY` on the laptop only). The answer is filtered and spoken as a `scene` message. It needs internet; local guidance continues without it. The image is processed in memory and never stored.

Part 5 – Backend Explanation

5.1 What the backend is made of

The backend is **one Python process** started by `neurosonic_app.py`. The web server uses only Python's standard library (`http.server.ThreadingHTTPServer`) – no Flask, Django or FastAPI.

Layer	Files	Job
Start-up	`neurosonic_app.py`, `START_NEUROSONIC.ps1`	read options, create every engine, start the servers
Web layer	`neurosonic/server.py`	HTTPS, routing, request size limits, JSON responses, static files
Engines	`awareness.py`, `detectors.py`, `pathguide.py`, `tracking.py`, `events.py`, `tts.py`, `voicecmd.py`, `maps.py`, `vision/`	the actual work
Research	`service.py` + `woad/`	WOAD replay, IR + depth upload, event log
Storage	`store.py`	SQLite history and saved places
Security / network	`tls.py`, `netcheck.py`	local certificate authority, Windows firewall check

5.2 How a request is handled

1. The phone opens a connection to port **8765**. `NeuroSonicHTTPServer` gives it its own thread (`ThreadingHTTPServer`, `daemon_threads = True`).
2. `finish_request()` performs the **TLS handshake in that worker thread** with a 10 s timeout (`TLS_HANDSHAKE_TIMEOUT`). One slow phone can therefore never freeze the whole server.
3. `do_GET` / `do_POST` call `_dispatch()`, which looks up the path in the `GET` or `POST` dictionary.
4. Unknown GET paths go to `_static()`, which serves files from `web/` and rejects path traversal (`../`).
5. The route function (`r_awareness_frame`, `r_tts`, …) does its work and replies with `_json()`.
6. Errors become proper HTTP codes: `ApiError` → its own code, bad input → **400**, voice engine failure → **503**, anything unexpected → **500**. A phone that disconnects is closed quietly.

5.3 All API endpoints

Method and path	Purpose	Handled by
`GET /api/status`, `GET /api/events?since=`	app status; new alert events	`service.py`
`POST /api/awareness/frame?session=&mode=&describe=`	**one camera JPEG → path state, events, boxes**	`awareness.py`
`GET /api/awareness/status`	detector, device, FP16/FP32, frame interval, fallback reason	`awareness.py`
`GET /api/tts?text=`, `GET /api/tts/status`	Piper neural voice as WAV (supports HTTP Range for iPhone)	`tts.py`

Method and path	Purpose	Handled by
`POST /api/voicecmd?lat=&lon=`, `GET /api/voicecmd/status`	16 kHz WAV → text → intent	`voicecmd.py`
`GET /api/map/info`, `/api/map/data`, `/api/map/search?q=`	offline map information, layers, place search	`maps.py`
`POST /api/route`	walking route with A*	`maps.py`
`GET /api/history`, `GET /api/stats`, `POST /api/history/clear`	alert history	`store.py`
`GET/POST /api/places`, `POST /api/places/delete`	saved places	`store.py`
`POST /api/alerts`	log a client-side alert (navigation, simulation)	`service.py`
`POST /api/voice/report`, `GET /api/voice/reports`	phone reports whether audio played	`server.py`
`GET /api/vision/status`, `POST /api/vision/describe?consent=1`	optional online description	`vision/assistant.py`
`POST /api/replay/start`, `/stop`, `GET /api/replay/state`, `/frame.jpg`	WOAD replay of recorded frames	`service.py` → `woad/`
`POST /api/detect`	one IR + depth pair → WOAD detections	`service.py` → `woad/`

A second, plain-HTTP server on port **8766** (`make_setup_server`) serves only the setup page and the local CA certificate, so the iPhone can trust the HTTPS server once.

5.4 Concurrency – who runs where

Thread	What runs on it
Main thread	`serve_forever()` accepts connections
One thread per connection	handshake, routing, JSON, Piper synthesis, Vosk recognition, routing
`neurosonic-infer` (exactly one)	every RF-DETR / SSDLite inference (`ThreadPoolExecutor(max_workers=1)`)
Replay thread (`service.py`)	WOAD replay frames
Setup server thread	port 8766

A lock (`self._lock`) and the single inference thread mean two phones never use the GPU model at the same time.

5.5 Data stored and privacy

`store.py` keeps **text only** in `data/neurosonic.sqlite3`: alert history (time, source, kind, sentence) and saved places. **No camera image is stored**: frames are decoded in memory and discarded. GPS positions stay on the phone; only the start and end points of a requested route are sent to the laptop.

Part 6 – Frontend Explanation

6.1 Technology

Plain **HTML, CSS and JavaScript modules** (`<script type="module">`), with no framework and no build step. The page is served by the laptop and runs in Safari on the iPhone.

File	Job
`web/index.html`	all screens: Home (camera awareness), Navigate, History, Settings
`web/style.css`	high-contrast dark theme, large buttons, iPhone safe areas
`web/js/app.js`	the controller: tabs, camera loop, voice commands, navigation, history
`web/js/speech.js`	`SpeechQueue` (priorities, de-duplication, stale-dropping) and `Announcer` (plays Piper WAV, browser voice as backup)
`web/js/nav.js`	GPS maths: distance, bearing, compass words, route following, off-route detection
`web/js/mapview.js`	draws the offline map and route on a canvas
`web/js/recorder.js`	records the microphone and converts it to 16 kHz mono WAV for Vosk
`web/js/eventsim.js`	simulated events for demonstrations and tests

6.2 Camera loop on the phone – app.js

1. **Start** → `startAwareness()` asks for the rear camera (`facingMode: "environment"`, 640 × 480).
2. `awarenessStep()` runs every `recommended_frame_ms` (200 ms GPU / 400 ms CPU):
3. skip while the page is hidden; after 4 s without a picture say "camera unavailable";
4. draw the video into a **320-pixel-wide canvas**, encode **JPEG quality 0.7**;
5. `POST /api/awareness/frame`, cancelled after **5 s** by an `AbortController`;
6. only one request is ever in flight; the next one is scheduled in `finally`.
7. With the answer: `drawAwareness()` draws the path trapezoid and boxes, `setPathState()` changes the large coloured card, and `voice.say(text, category, …)` speaks the events.
8. Three errors in a row → "Cannot reach the laptop for camera analysis".

6.3 Speech priorities – speech.js

Category	Priority	Interrupts?	Dropped if older than
`urgent` ("Stop")	4	yes	2.5 s
`system`	3	yes	15 s
`obstacle`	2	yes	4 s
`motion`, `navigation`, `scene`	1	no	3 s / 15 s / 30 s
`info`, `description`	0	no	10 s / 3 s

Messages with the same `key` (e.g. `"path"`) replace each other in the queue, so an out-of-date "Caution" is never spoken after the path became clear. Fixed warning sentences are fetched in advance so "Stop" is not delayed.

6.4 Accessibility features in the code

- Screen-reader labels (`aria-label`) on every section and button; large touch targets.
- Everything important is **spoken**, not only shown.
- High-contrast colours: green = appears clear, orange = caution, red = stop, grey = uncertain.

Part 7 – GPU and Python Environment

7.1 Hardware

Item	Value
GPU	NVIDIA GeForce RTX 4050 Laptop GPU, 6 GB (6141 MiB), driver 592.82
CPU / RAM	AMD Ryzen 7 7840HS (8 cores / 16 threads), about 16 GB
OS	Windows 11

7.2 Two Python environments (both Python 3.12.10)

	`.venv-gpu` **(normal use)**	`.venv` **(CPU-only)**
PyTorch	`2.15.0+cu132` (CUDA 13.2)	`2.15.0+cpu`
torchvision	`0.29.1+cu132`	`0.29.2`
Extra	`rfdetr 1.11.2`	–
`torch.cuda.is_available()`	**True**	False
Camera detector	RF-DETR Nano, GPU, FP16	SSDLite, CPU
Started by	`START_NEUROSONIC.ps1`	`START_NEUROSONIC.ps1 -Cpu`

Shared packages (from `requirements.txt`): NumPy 2.5.4, OpenCV 5.0, piper-tts 1.8.0, onnxruntime 1.31.1, vosk 0.3.45, google-genai 2.34.0 (optional).

7.3 How to check the GPU in front of the faculty

```
..\.venv-gpu\Scripts\python -c "import torch; print(torch.__version__, torch.cuda.is_available(), torch.cuda.get_device_name(0))"
# 2.15.0+cu132 True NVIDIA GeForce RTX 4050 Laptop GPU
nvidia-smi          # shows the python.exe process using GPU memory while the app runs
```

In the app: open `https://<laptop-ip>:8765/api/awareness/status` and show `"device": "cuda"`, `"precision": "fp16"`, `"recommended_frame_ms": 200`.

7.4 Two GPU details worth mentioning

- **CPU threads limited to 2** while the GPU runs: PyTorch's default 8 idle threads were spinning, measured 712 % of one core vs 121 % with 2 threads, at the same latency.
- **Low-power state:** at 5 frames per second the laptop GPU drops to its lowest clock between frames, so paced latency (53 ms FP16) is higher than a back-to-back benchmark (17 ms). FP16 reduces this penalty.

Part 8 – Testing and Benchmarking

8.1 Automated tests (summary of Part 3, F5)

Kind	Count / result	How
Python unit and integration tests	**204 tests, OK** (3 Nov 2026) in `.venv-gpu` (1 skipped: live Gemini) and `.venv` (3 skipped: GPU-only)	`python -m unittest discover -s tests -t .`
JavaScript logic tests	run inside the Python suite through Node.js (`test_web_js.py`)	speech queue, navigation maths, GPS states
Browser tests	**7 of 7 pass**	headless Chrome, iPhone screen size, fake camera and microphone
Offline checks	several tests forbid every non-local network connection	proves voice, map and routing work offline in-process

8.2 Benchmarks and what they proved

Benchmark (tool)	Data	Key result
Detector choice (`benchmark_detectors.py`)	365 near-IR validation images, 129 people	RF-DETR Nano person AP@0.5 **0.660** vs SSDLite 0.542; RF-DETR 21.3 ms on GPU (FP32)
FP16 vs FP32 (`benchmark_precision.py`)	same images; 320×240 frames paced at 5 fps	same accuracy (0.6600 vs 0.6597); median **53.2 vs 66.5 ms**
Stability (`soak_test.py`)	demo clip, 10 minutes, 5 fps	2,980 frames, **0 errors**, 4.97 fps, flat memory after the one-thread fix
End-to-end in browser	laptop Chrome, fake camera	camera → answer median **67 ms**; event → speech about 260 ms
WOAD reproduction (`evaluate.py`, `verify_equivalence.py`)	published WOAD dataset	copy identical to original (max difference 0.0); person AP@0.5 **0.218**

8.3 Detector comparison (Phase 1, RTX 4050)

Detector	Device	Median latency	Person AP@0.5	Recall @ 0.40	Licence
SSDLite320	CPU	43.7 ms	0.542	0.574	BSD-3
RF-DETR Nano	**GPU**	**20.6 ms**	**0.660**	**0.729**	Apache-2.0
YOLO26n	GPU	11.8 ms	0.619	0.628	AGPL-3.0
YOLO26s	GPU	11.4 ms	0.628	0.651	AGPL-3.0

Why RF-DETR and not the faster YOLO26? Highest accuracy and recall (missing a person is the worst error for a blind user), fast enough for 5 frames per second, and a permissive licence.

8.4 What is NOT yet tested (say this honestly)

- No test on the **physical iPhone** while walking; no field test on Vellore streets.

- No test with **blind or low-vision users**; thresholds are engineering defaults.
- No test with the Wi-Fi physically disconnected (only in-process network blocking).
- Accuracy numbers come from near-IR images, the only labelled data available, not from phone photos.
- Soak test was 10 minutes; a run of an hour or more is pending.

Part 9 – Important Code Snippets Explained Line by Line

9.1 Server start-up – `neurosonic_app.py`

```
store = Store(a.db)                                          # 1
service = DetectionService(store, a.weights, a.device)       # 2
tts = PiperTTS(a.voice_model, cache_dir=ROOT / "data" / "tts_cache")
if tts.warm_up(): ...                                        # 3
awareness = RGBAwareness(a.rgb_weights,
        detector=a.detector or os.environ.get("NEUROSONIC_DETECTOR") or "auto",
        events=not a.no_events)                              # 4
if awareness.warm_up(): ...                                  # 5
httpd = make_server(service, store, a.host, a.port, ssl_context=ctx, tts=tts,
        offline_map=offline_map, voicecmd=voicecmd, awareness=awareness, vision=vision)  # 6
```

1. Open (or create) the SQLite database for history and places.
2. Load the WOAD checkpoint for the research screens (hash-checked).
3. Load the Piper voice; if it fails the app still runs and the phone uses its built-in voice.
4. Create the camera pipeline. The detector choice comes from `--detector`, then the environment variable, else `auto`.
5. Warm-up runs one detection on a black image, so the first real frame is not slow and errors appear at start-up.
6. Every engine is handed to the server ("dependency injection"); the server itself creates nothing.

9.2 Routing – `server.py` `_dispatch`

```
url = urlparse(self.path)                  # 1  split "/api/tts?text=hi" into path and query
fn = routes.get(url.path)                  # 2  look up the handler in the GET or POST table
if fn is None:
    if self.command in ("GET", "HEAD"):
        return self._static(url.path)      # 3  not an API path: serve a file from web/
    raise ApiError(404, "not found")
fn(self, url)                              # 4  run the handler
except (InvalidInput, ValueError) as e:
    self._json({"error": str(e)}, 400)     # 5  bad input -> clear error, server keeps running
```

9.3 Camera frame endpoint – `server.py` `r_awareness_frame`

```
jpg = self._raw_body(3 * 1024 * 1024, {"image/jpeg", "application/octet-stream"})   # 1
session = (q.get("session", ["default"])[0] or "default")[:64]                       # 2
res = awareness.process_frame(jpg, session=f"{self.client_address[0]}:{session}",
                              mode=mode, describe=describe)                          # 3
for e in res["events"]:
    if e["kind"] in ("object", "path", "event"):
        service.events.add("awareness", e["kind"], e["text"], e["priority"], e["data"], "camera")  # 4
self._json(res)                                                                      # 5
```

1. Read the raw JPEG body, at most 3 MB and only allowed content types.
2. The session id from the phone, cut to 64 characters.

3. Session key = phone IP + session id, so two phones never share tracking history.
4. Each spoken event is also written to the event log and the SQLite history.
5. Send the result as JSON.

9.4 One inference thread – `awareness.py`

```
self._infer = ThreadPoolExecutor(max_workers=1, thread_name_prefix="neurosonic-infer")   # 1

def detect(self, bgr):
    return self._infer.submit(self._detect_on_worker, bgr).result()                     # 2

def _detect_on_worker(self, bgr):
    with self._lock:                                                                     # 3
        m = self._load()                                                                 # 4
        dets = m.detect(bgr)                                                             # 5
```

1. One long-lived worker thread for all model work (fixed the ~2 MiB/s memory growth).
2. The request thread hands the image over and waits for the answer.
3. The lock makes sure only one inference runs at a time.
4. Load the detector on first use (`make_detector`: RF-DETR or SSDLite).
5. Run it; the result is a list of `{class_id, class_name, confidence, box_xyxy}`.

9.5 GPU detector with FP16 – `detectors.py`

```
if device == "cuda" and not torch.cuda.is_available():
    raise DetectorUnavailable(...)                        # 1 no GPU -> caller falls back to SSDLite
os.environ.setdefault("HF_HUB_OFFLINE", "1")              # 2 never download anything
torch.set_num_threads(GPU_MODE_CPU_THREADS)               # 3 2 CPU threads are enough
self.model = RFDETRNano(device=device, pretrain_weights=str(weights))   # 4
try:
    self.model.inference(compile=False, dtype=torch.float16)            # 5
    self.precision = "fp16"
except Exception:
    pass                                                  # 6 keep FP32 if FP16 is unsupported
```

9.6 Automatic fallback during a session – `FallbackDetector.detect`

```
try:
    return self.active.detect(bgr)                        # 1 normal case
except Exception as e:
    if isinstance(self.active, SSDLiteDetector):
        raise                                             # 2 SSDLite itself failed: report it
    self.fallback_reason = f"{self.active.name} failed during inference ..."
    torch.set_num_threads(self.active.prev_threads)       # 3 give SSDLite all CPU threads again
    self.active = self._fallback_factory()                # 4 switch permanently to SSDLite
    return self.active.detect(bgr)                        # 5 answer the SAME frame
```

9.7 Phone side – `app.js` `awarenessStep`

```
const w = 320, h = Math.round((v.videoHeight / v.videoWidth) * 320);   // 1
aCanvas.getContext("2d").drawImage(v, 0, 0, w, h);
const blob = await new Promise((r) => aCanvas.toBlob(r, "image/jpeg", 0.7));   // 2
const kill = setTimeout(() => ctl.abort(), 5000);                       // 3
const res = await fetch(`/api/awareness/frame?session=${aSession}&mode=${settings.mode}...`,
  { method: "POST", headers: { "Content-Type": "image/jpeg" }, body: blob, signal: ctl.signal });  // 4
for (const e of r.events) {
  if (e.kind === "path") voice.say(e.text, e.category, null, { key: "path" });              // 5
}
...
finally { scheduleFrame(Math.max(0, frameMs() - (Date.now() - t0))); }                    // 6
```

1. Scale the video frame to 320 px wide, keeping the aspect ratio (small upload, fast on Wi-Fi).
2. Compress to JPEG, quality 0.7 (small enough for Wi-Fi at 5 frames per second).
3. If the laptop does not answer within 5 s, cancel – a stuck request must not stop guidance.
4. Send the image to the laptop.
5. Speak each path message; `key: "path"` replaces any older path message still waiting.
6. Schedule the next frame so that frames are evenly spaced (200 ms minus the time already used).

Part 10 – Faculty Viva Questions and Answers

10.1 Project and architecture

Q1. What problem does NeuroSonic solve? It gives a blind or low-vision walker spoken, calm information about the walking path ahead (clear, caution, stop), people or vehicles moving into the path, offline walking directions, and an optional scene description. It is a research prototype, not a safety device.

Q2. Why does the phone not run the AI itself? The laptop GPU runs RF-DETR in about 53 ms per frame. The phone is a thin client: camera, microphone, GPS, speaker and display. This keeps the phone app a plain web page with no install.

Q3. Why a web app instead of a native iOS app? No App Store, no Mac needed to build it, and the same code runs on any phone. iPhones allow camera, microphone and GPS on HTTPS pages, which is why `tls.py` creates a local certificate authority.

Q4. Which framework is the backend? Python's standard library `http.server` (`ThreadingHTTPServer`). No Flask or FastAPI – fewer dependencies and full control over TLS handshakes and timeouts.

Q5. Does it need internet? No, except for the optional Gemini description. Detection, voice, speech recognition, maps and routing all run on the laptop.

10.2 AI and models

Q6. Which model detects obstacles? RF-DETR Nano (COCO-pretrained) on the RTX 4050 in FP16, with SSDLite320- MobileNetV3 on the CPU as automatic fallback.

Q7. Did you train a model? No. Both use public COCO weights. We benchmarked candidates and chose by accuracy, speed and licence.

Q8. What is the difference between RF-DETR and YOLO/SSD? RF-DETR is a detection transformer: object queries each predict one object, so no NMS is needed. SSD and YOLO predict many overlapping candidate boxes and remove duplicates with non-maximum suppression.

Q9. What is FP16 and does it reduce accuracy? 16-bit floating point. In our test the person AP@0.5 was 0.6600 (FP16) vs 0.6597 (FP32), and paced latency dropped from 66.5 to 53.2 ms.

Q10. What happens if the GPU is not available? `make_detector()` falls back to SSDLite on the CPU and the phone sends a frame every 400 ms instead of 200 ms. If RF-DETR fails mid-session, `FallbackDetector` switches and re-analyses the same frame.

Q11. What is the confidence threshold? 0.40 for both detectors.

Q12. Can it measure distance? No. A single RGB camera has no depth; the app never states a distance.

Q13. Can it see steps, kerbs and potholes? No. COCO has no such classes. This is a stated limitation.

Q14. What is the base paper's model and why isn't it used live? WOAD fuses near-infrared and depth images with a transformer. It needs the CS30 sensor, not a phone camera. We reproduced it on Windows (identical output) and measured person AP@0.5 0.218 on the published data.

10.3 Algorithms

Q15. How do you decide "path blocked"? A trapezoid in the lower centre of the image is the path. If an object covers ≥ 30 % of it (centred) in 2 of the last 3 frames, the state is BLOCKED. CLEAR needs 3 clear frames.

Q16. Why not announce every object? Too much speech is dangerous and tiring. Only state changes and specific events are spoken, with priorities and cooldowns.

Q17. How do you separate the user's own movement from object movement? `tracking.py` tracks background corner points with Lucas–Kanade optical flow and fits a similarity transform with RANSAC; object motion is measured relative to that.

Q18. What tracker do you use? A small own implementation of the ByteTrack idea: match confident detections first, then low-confidence ones to existing tracks, by IoU within the same class.

Q19. What is IoU? Intersection over union – overlap area divided by the combined area of two boxes.

Q20. How does navigation work offline? `maps.py` loads a local OpenStreetMap extract of Vellore and runs A* on the walking graph; `nav.js` turns GPS positions into spoken turn instructions.

10.4 Engineering and testing

Q21. What was the hardest bug? Server memory grew about 2 MiB/s because the GPU model ran on ever-new request threads. Moving all inference onto one dedicated thread fixed it (10-minute soak: 0 errors, flat memory).

Q22. How do you know it works? 204 automated tests, 7 browser tests with a fake camera, benchmarks and a soak test. Physical iPhone and street tests are still pending.

Q23. How is privacy protected? Camera images are processed in memory and never stored; history keeps text only; GPS stays on the phone; the Gemini key stays on the laptop; online description needs explicit consent.

Q24. What would you do next? Field tests with the iPhone, tests with visually impaired users, a model trained for kerbs, steps and poles, and depth estimation for distance.

Part 11 – Quick Revision Sheet

11.1 Numbers to remember

Item	Value
Live detector	**RF-DETR Nano**, RTX 4050, **FP16**, 384 px input
Fallback	**SSDLite320-MobileNetV3**, CPU, 320 px
Confidence threshold	**0.40**
Frame interval	**200 ms GPU** (5 fps) / 400 ms CPU
Phone upload	**320 px wide JPEG, quality 0.7**, 5 s timeout, one request at a time
Latency	RF-DETR paced **53.2 ms** (FP32 66.5); camera → answer median **67 ms**
Accuracy (near-IR set)	person AP@0.5 RF-DETR **0.660**, SSDLite 0.542
Path states	CLEAR_VIEW, POSSIBLE_OBSTACLE, BLOCKED_VIEW, UNCERTAIN, CAMERA_UNAVAILABLE
Confirmation	BLOCKED 2 of 3 frames; CLEAR 3 frames
Tests	**204** Python tests OK, **7/7** browser tests
Soak test	10 min, 2,980 frames, **0 errors**
WOAD reproduction	output identical (diff 0.0); person AP@0.5 **0.218**
Ports	**8765** HTTPS app, **8766** setup page
Environment	Python 3.12.10, torch 2.15.0+cu132, CUDA 13.2, rfdetr 1.11.2

11.2 The pipeline in one line

Phone camera → 320 px JPEG → HTTPS POST `/api/awareness/frame` → `server.py` → `awareness.py` (quality check → RF-DETR on GPU → tracking with camera-motion compensation → path state → events) → JSON → `app.js` draws and `speech.js` speaks with Piper voice from `/api/tts`.

11.3 Which file does what

If asked about ...	Show this file
Start-up	`neurosonic_app.py`, `START_NEUROSONIC.ps1`
API / server	`neurosonic/server.py`
Camera pipeline	`neurosonic/awareness.py`
AI model, GPU, FP16	`neurosonic/detectors.py`
Clear / caution / stop	`neurosonic/pathguide.py`
Moving objects	`neurosonic/tracking.py`, `neurosonic/events.py`
Voice out / voice in	`neurosonic/tts.py` (Piper), `neurosonic/voicecmd.py` (Vosk)
Maps and routing	`neurosonic/maps.py`, `web/js/nav.js`
History	`neurosonic/store.py`

If asked about ...	Show this file
HTTPS on iPhone	`neurosonic/tls.py`
Online description	`neurosonic/vision/assistant.py`, `gemini.py`
Base paper	`woad/`
Phone UI	`web/index.html`, `web/js/app.js`, `web/js/speech.js`
Tests / benchmarks	`tests/`, `tools/`

11.4 Honest limitations (always say them)

1. Not a safety device; does not replace a cane or guide dog.
2. No distance; no steps, kerbs, potholes, poles or walls.
3. Forward camera view only, not 360 degrees.
4. Thresholds not validated with users; no iPhone field test yet.
5. Accuracy measured on near-IR images, not phone photos.

11.5 Demo checklist

1. `START_NEUROSONIC.ps1` → log shows `Camera awareness: RF-DETR Nano (Roboflow, COCO, 384 px) on the NVIDIA GPU ready (cuda; frame interval 200 ms; …)`.
2. iPhone Safari → `https://<laptop-ip>:8765` → Start → point at a person → "Caution" / "Stop" is spoken.
3. Show `/api/awareness/status`: `cuda`, `fp16`.
4. `nvidia-smi` shows python.exe on the GPU.
5. Speak "navigate to ..." → route is spoken.
6. History tab shows the spoken alerts.